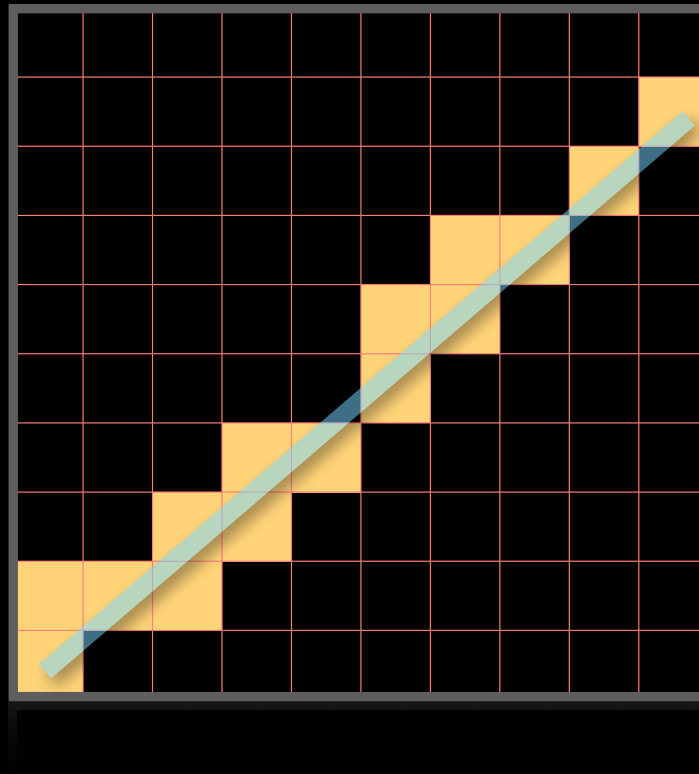


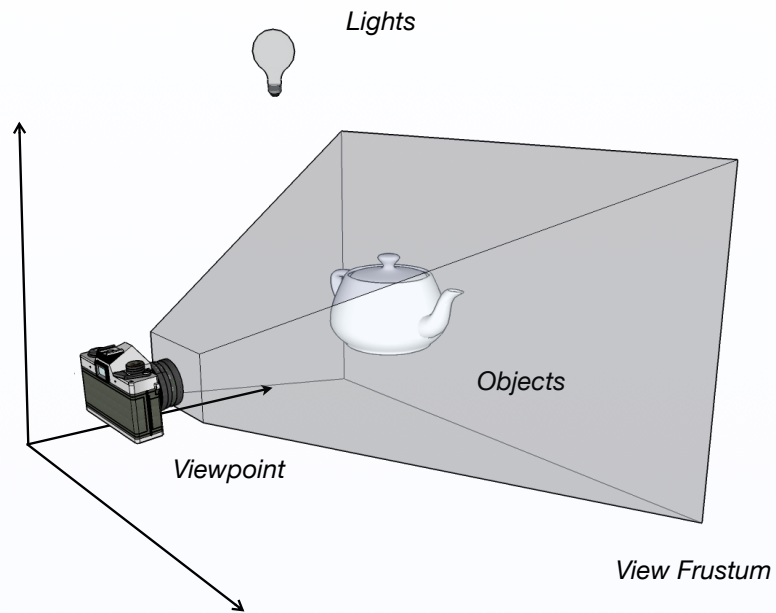
Die kürzeste
Strecke
zwischen zwei
Punkten ist nicht
eine Linie



Eine Linie Zeichnen



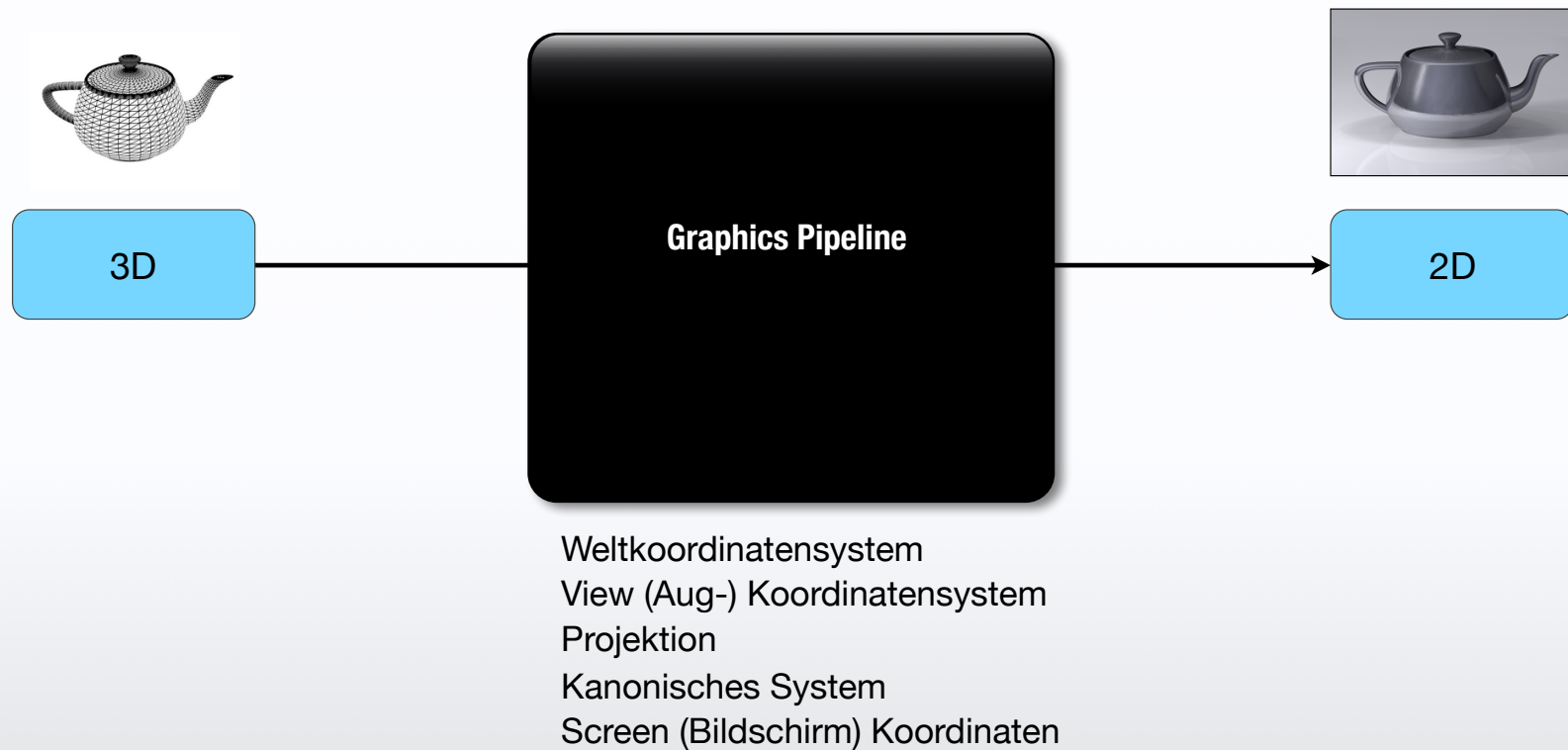
from 3D to 2D

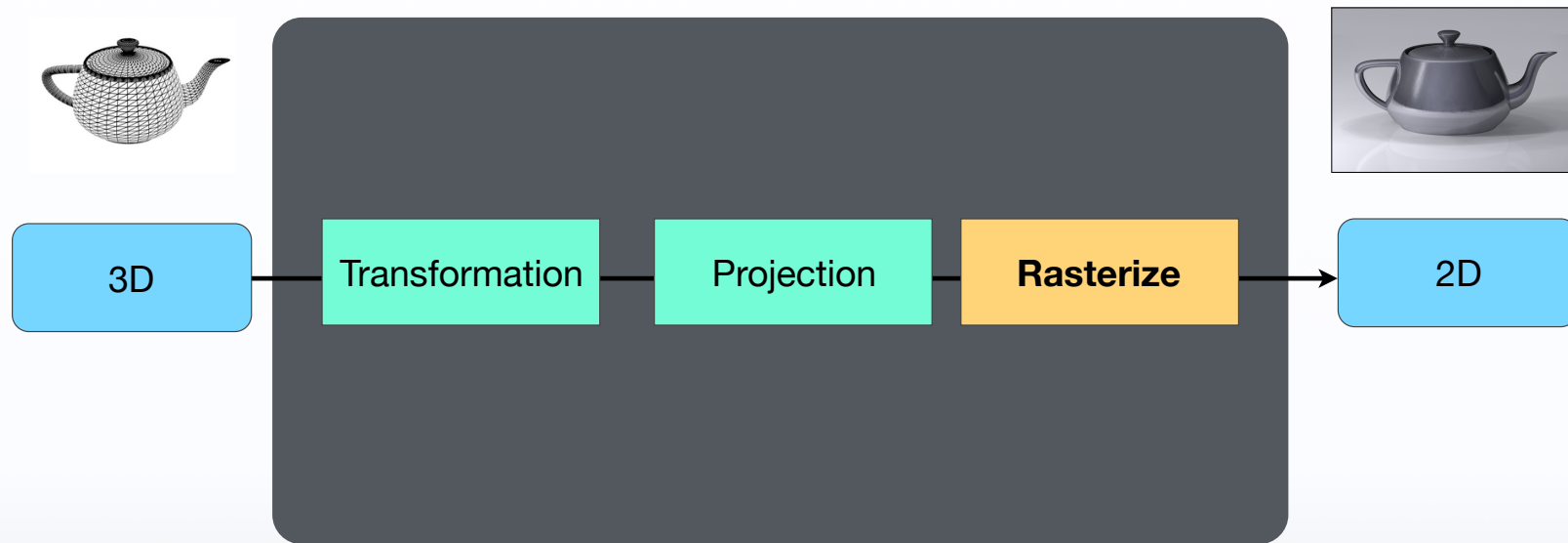


3D scene in world coordinates

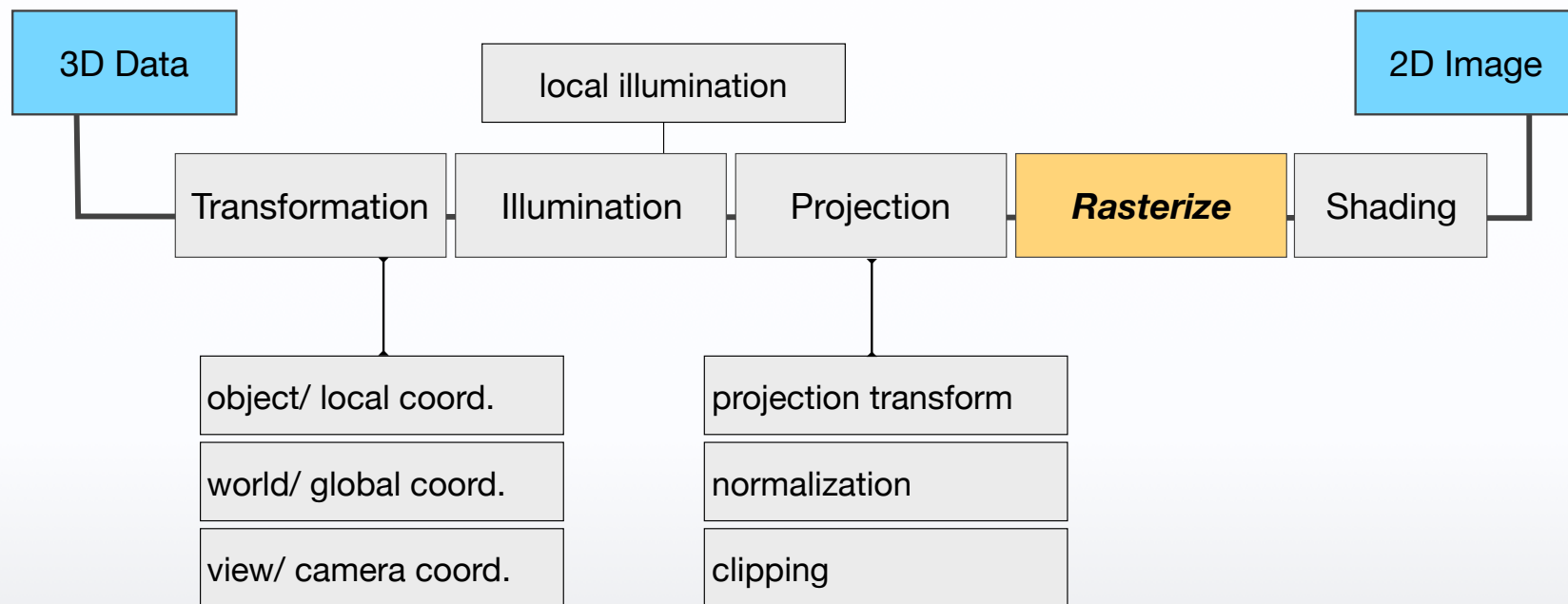


screen in pixel coordinates





Fixed Graphics Pipeline



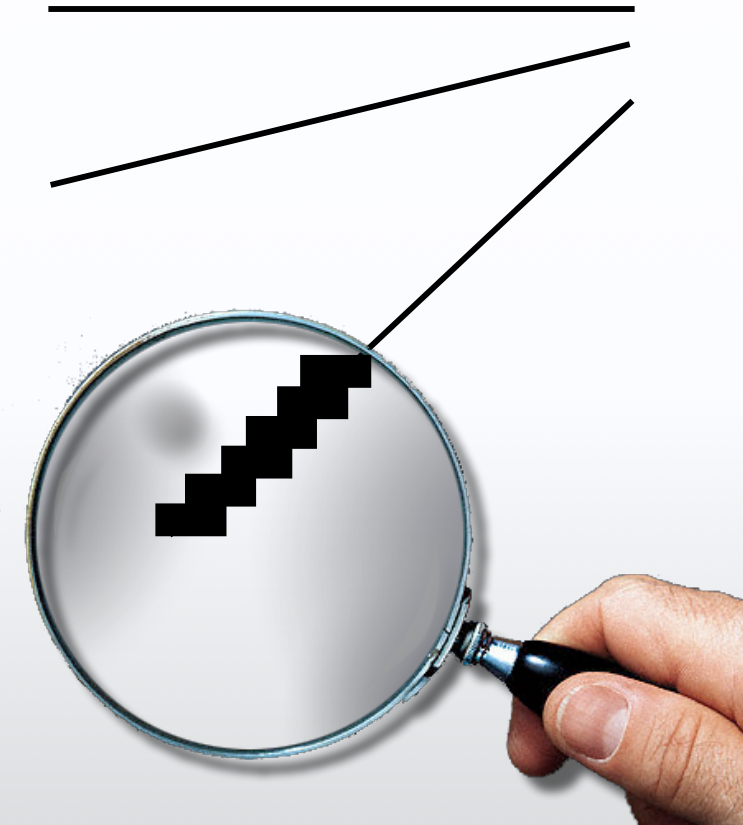
Rastering

- Ausgabegerät (Bildschirm) mit endlichem festem Raster
- Auf der Grafikkarte gibt es einen eigenen Bildspeicher (Framebuffer)
- Die Werte werden ausgelesen und weitergegeben; oder analog umgewandelt.
- z.B. Format: RGBA 4 Byte Pro Pixel
- Farbwert + Alpha
- Problem der Rasterisierung:
 - Näherung einer Linie durch kleine Flächenelemente



Eine Linie

- Flächenelement: Pixel
 - “Picture Element”
 - kleinste Einheit des Rasterbildes
 - gleichförmig in Zellen & Spalten
- gesucht: gerade Linie
- Forderungen:
 - gerade aussehen
 - gleichmässig hell (in alle Richtungen)
 - gleichmässig dick
 - Anfangs- und Endpunkte exakt
 - Schneller Algorithmus !!!
 - keine Treppen



Linie - parametrische Beschreibung

- Einführung einer Parametrisierung t

- $x = x_1 + t(x_2 - x_1)$

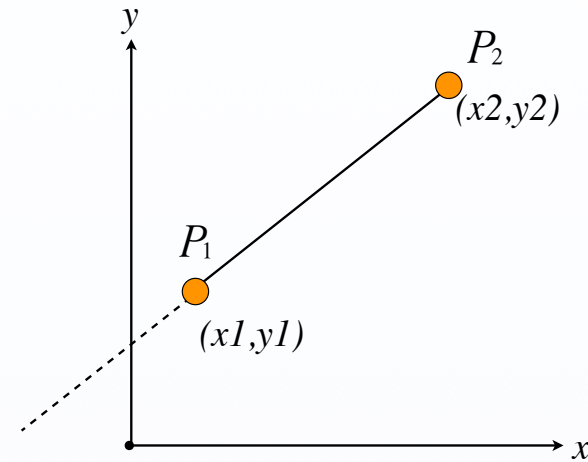
- $y = y_1 + t(y_2 - y_1)$

- $t \in [0, 1]$

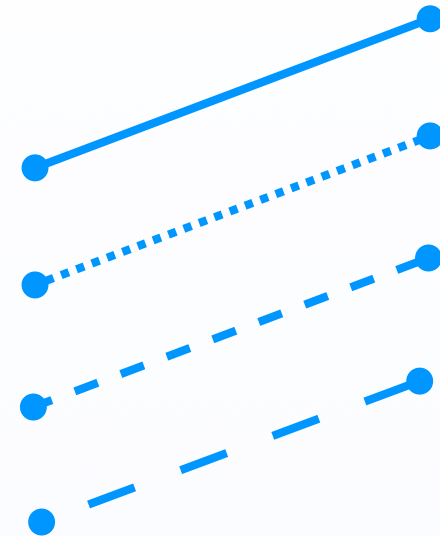
- Vektoriell

- $x = x_1 + t \cdot \vec{v}_x$

- $y = y_1 + t \cdot \vec{v}_y$



```
void draw_line(int x1, int y1, int x2, int y2){  
    float x,y;  
    float t;  
    float step=0.01;  
  
    for (float t=0.0; t<=1.0 ; t=t+step) {  
        x = x1+t*(x2-x1);  
        y = y1+t*(y2-y1);  
        setPixel(x,y); // kein regulärer Befehl  
    }  
}
```



- ? Nachteile:
- Parameterdarstellung nicht geeignet
 - Wie groß sollte step sein ?
 - Y-Wert wird berechnet; X-Wert wird berechnet
 - Viele float Operationen
 - komplizierte Rechnungen in der Schleife

Linie - explizite Form

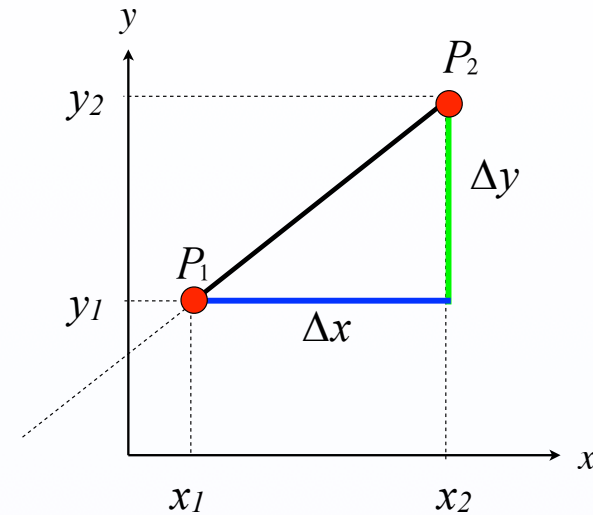
- Geradengleichung
- gegeben zwei Punkte $P_1(x_1, y_1)$ und $P_2(x_2, y_2)$
- gesucht: Beschreibung der Linie dazwischen:
- Steigung:
 - $\Delta x = x_2 - x_1$ ■
 - $\Delta y = y_2 - y_1$ ■
 - $m = \frac{\Delta y}{\Delta x}$
- Koordinatenform:

$$y = m \cdot x + n$$

Steigung

? Was ist die Bedeutung von n

? Wie wird der Wert berechnet



Berechnung Achsenabschnitt

Gegeben: $(x_1, y_1)(x_2, y_2)$

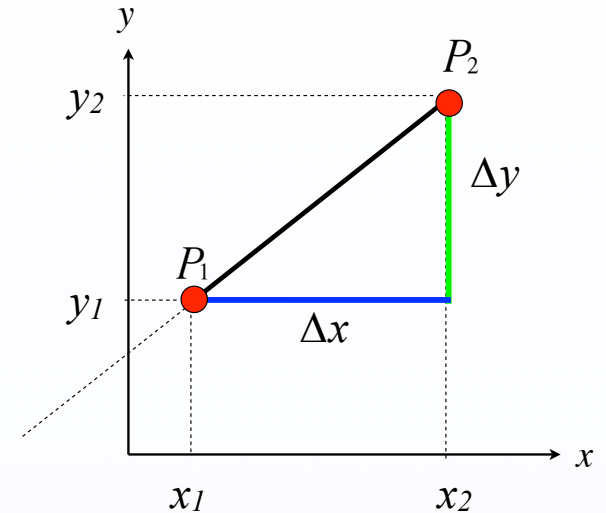
$y = m \cdot x + n$ Geradengleichung, damit auch

$$y_1 = m \cdot x_1 + n$$

$$n = y_1 - \frac{y_2 - y_1}{x_2 - x_1} \cdot x_1$$

$$n = \frac{y_1 x_2 - y_1 x_1 - y_2 x_1 + y_1 x_1}{x_2 - x_1}$$

$$n = \frac{y_1 x_2 - y_2 x_1}{x_2 - x_1}$$



Linie (mathematisch)

- Berechnung Achsenabschnitt: $n = \frac{y_1x_2 - y_2x_1}{x_2 - x_1}$
 - Für jeden Wert x_i
 - mit $y_i = m \cdot x_i + n$
 - nur den y_i -Wert berechnen
 - Berechneten Wert ganzzahlig abrunden, y_i'
 - Pixel ist: (x_i, y_i')
- Code:

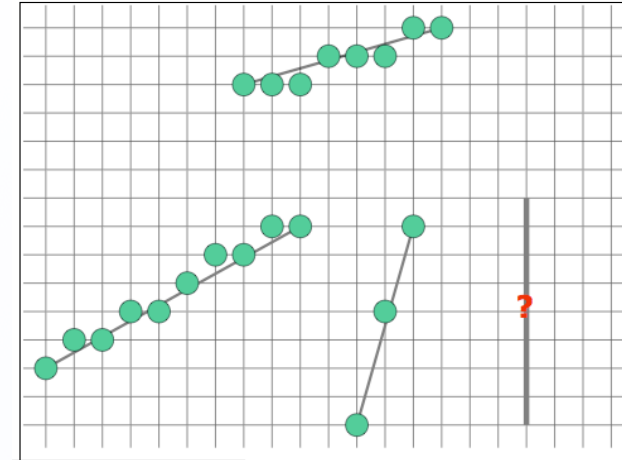
```
void draw_line(int x1, int y1, int x2, int y2){
    float m= (float)(y2-y1)/(x2-x1);
    float n= (y1*x2-y2*x1)/(x2-x1);

    for (float x=x1; x <= x2 ; x++) {
        float y=m*x+n;
        setPixel(x,y);
    }
}
```

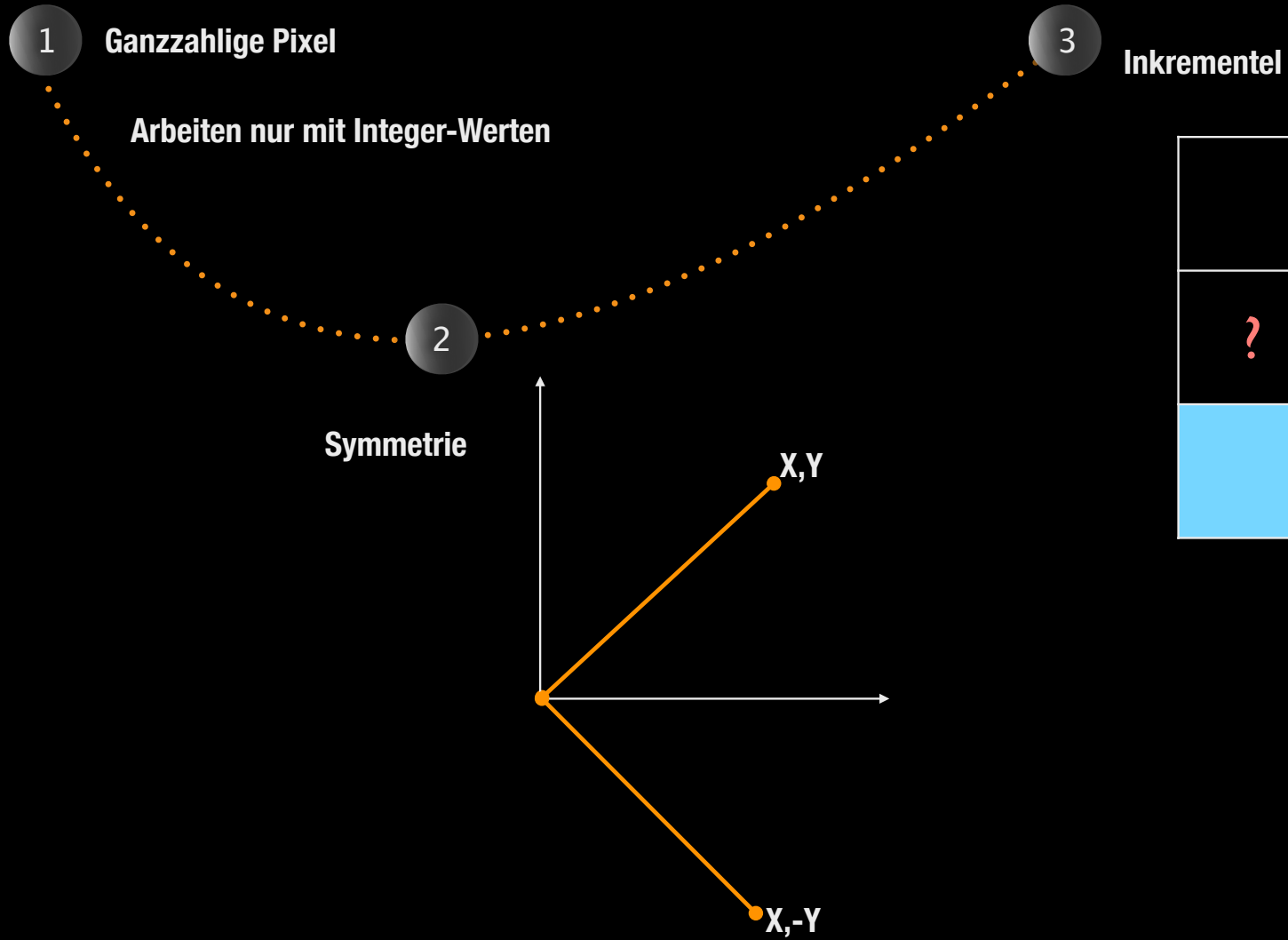
? wofür (float)(y...

Nachteile

- Bei Steigungen grösser 1 keine optisch zusammenhängende Linie, da y-Werte zu weit auseinander liegen
- Startpunkt exakt, Endpunkt gerundet
- Unterschiedliche Striche bei gleichen Endpunkten je nach Startpunkt
- Jedes mal eine Multiplikation nötig
- Jedes mal eine Fließkommaoperation nötig
- Berechnungen bauen nicht aufeinander auf



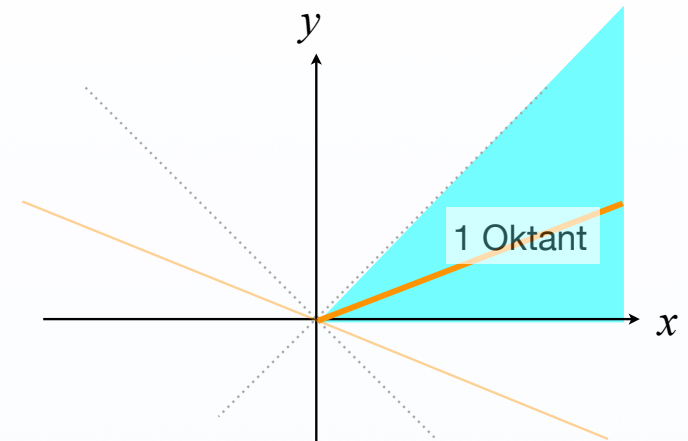
```
for (float x=x1; x <= x2 ; x++){  
    float y=x*m+n;  
    setPixel(x,y);  
}
```



?	?	
	?	

Neue Ansätze

- Inkrementell arbeiten !
- Einsatz von Integer-Arithmetik
 - Pixel liegen sowieso auf Integer-Gitter
 - kein float verwenden
- Keine Division & komplizierte Multiplikation
- Nur einen Oktanten betrachten (Symmetrie)
- Andere Oktanten
 - Vorzeichenwechsel / Anfangs- und Endpunktvertauschung
- Beim ersten Oktanten gilt:
 - Erhöht sich x um 1 kann sich y auch maximal nur um 1 erhöhen !
 - Steigung $0 \leq m \leq 1$
 - Achsenabschnitt verläuft durch den Ursprung
- Die Verschiebung anhand von Transformationen



Digital Differential Analyzer (Inkrementel)

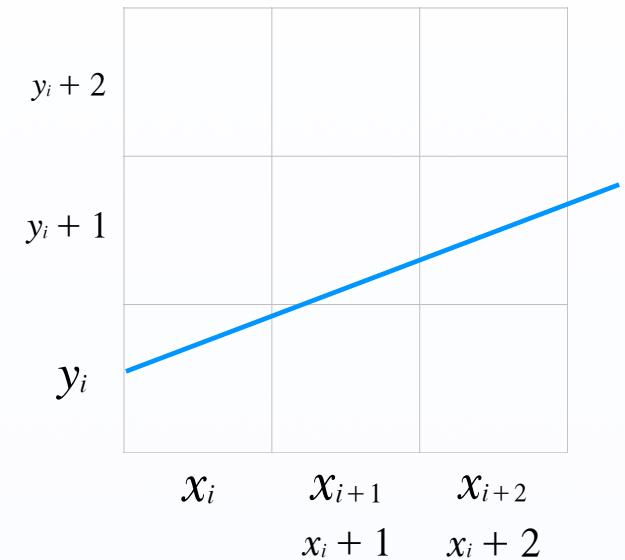
- Aus der Geradengleichung:

- $y_i = m \cdot x_i + n$

- inkrementeller Ansatz:

- $y_{i+1} = m \cdot x_{i+1} + n$
 $= m(x_i + 1) + n$ In unserem Fall
 $= m \cdot x_i + n + m$
 $= y_i + m$

- Für alle Geraden mit $0.0 < m < 1.0$!
- Linien werden von links nach rechts gezeichnet !
- Bei Steigung $m > 1.0$
- $x_{k+1} = x_k + \frac{1}{m}$

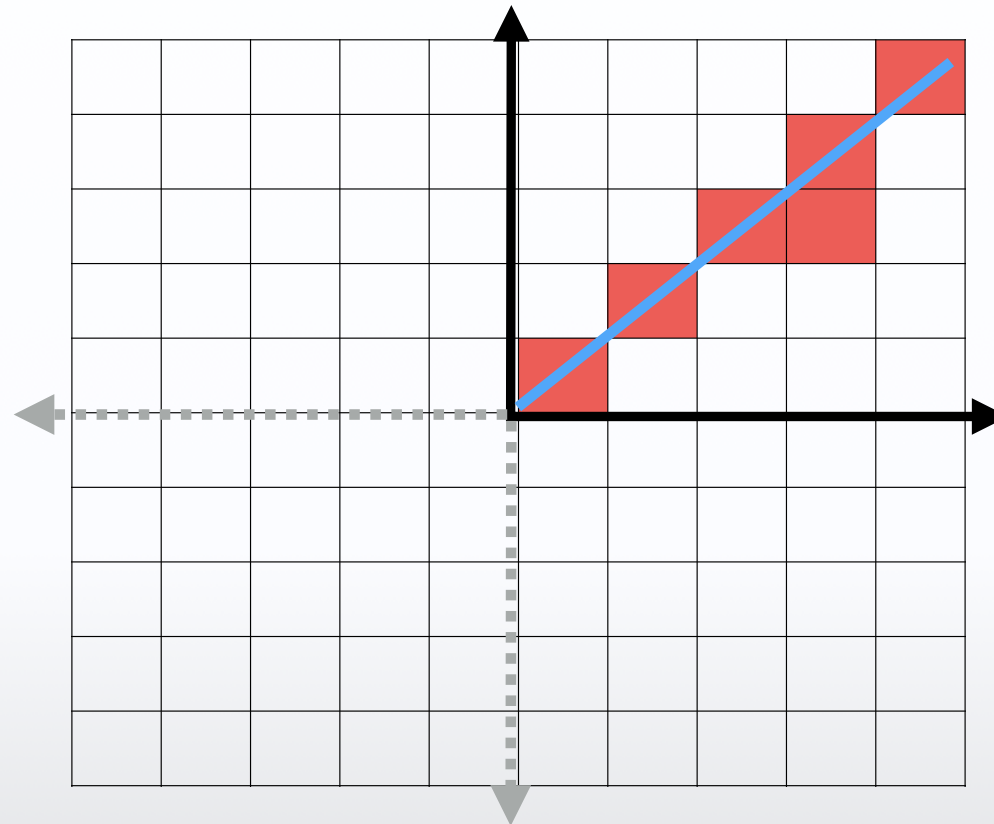


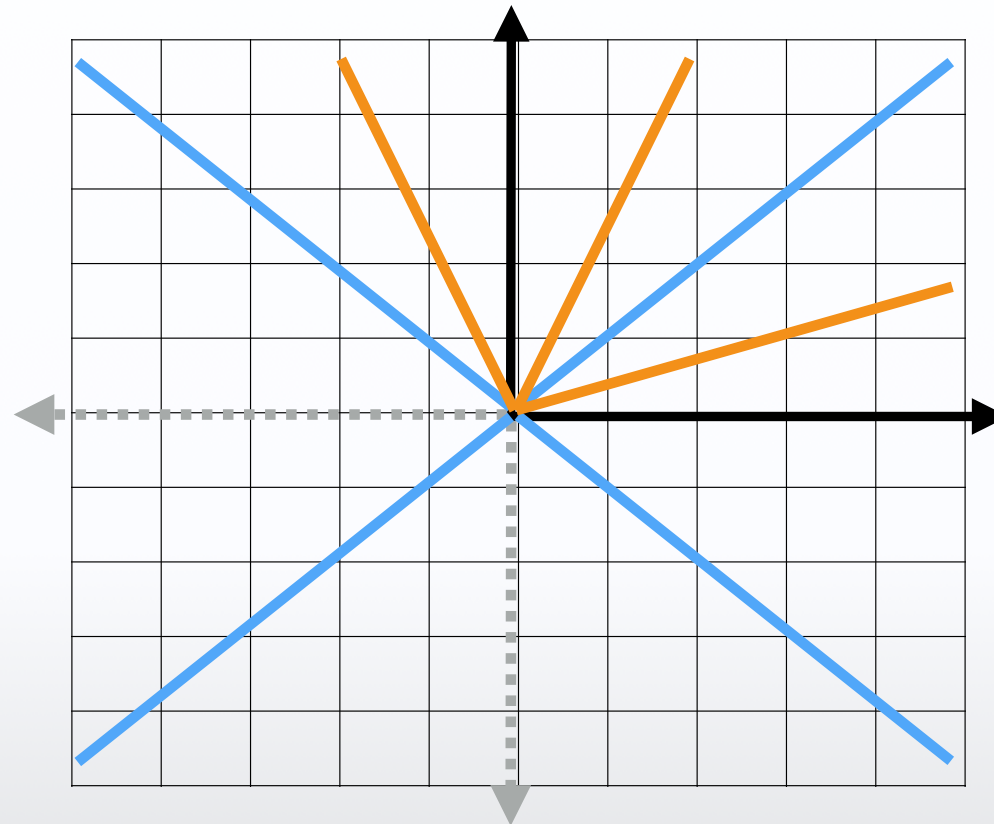
```
void draw_line(int x1, int y1, int x2, int y2){  
    float m= (y2-y1)/(x2-x1);  
    float n= (y1*x2-y2*x1)/(x2-x1);  
  
    for(float x=x1; x <=x 2 ; x++) {  
        setPixel(x,y);  
        float y=m*x+n;  
    }  
}
```

```
void draw_line(int x1, int y1, int x2, int y2){  
    float y=y1;  
    float m= float (y2-y1)/(x2-x1);  
  
    for( int x=x1; x <= x2 ; x++) {  
        setPixel(x,y);  
        y+=m;  
    }  
}
```

Eine Multiplikation gespart, aber immer noch Fließkomma Arithmetik

Nur ganze Zahlen

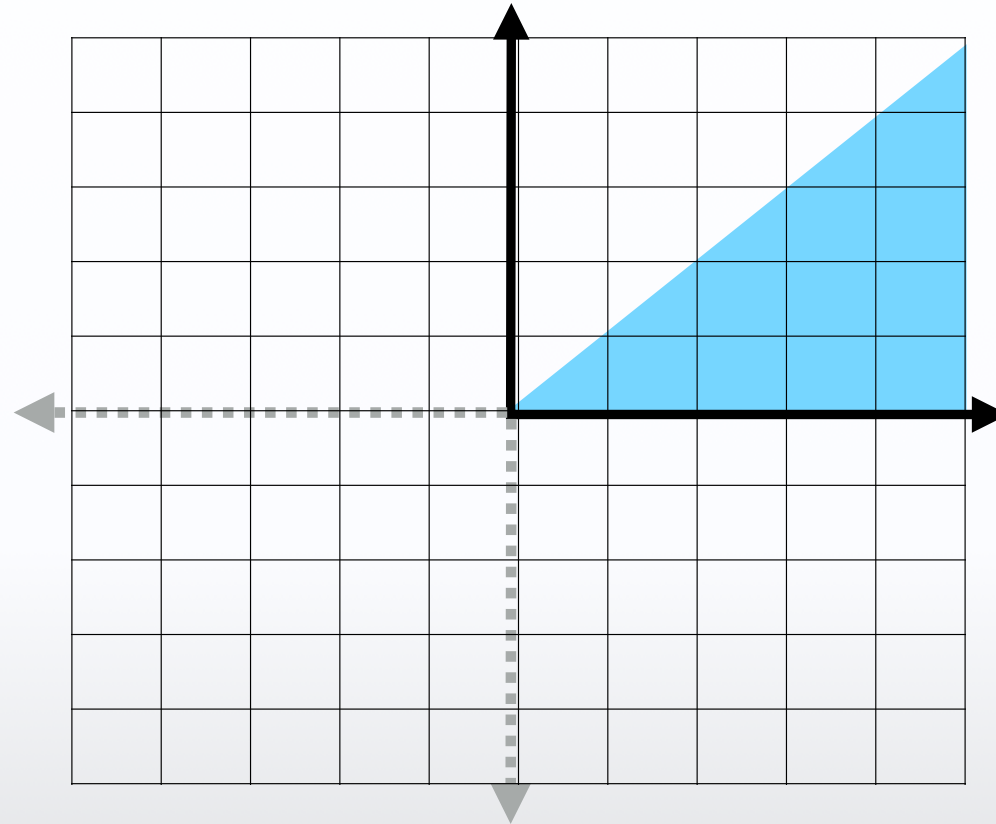




Fall 1 Oktant

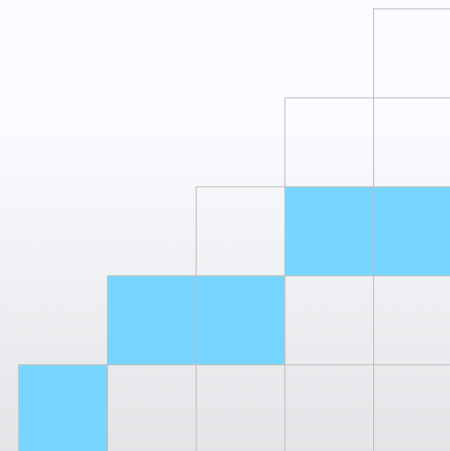
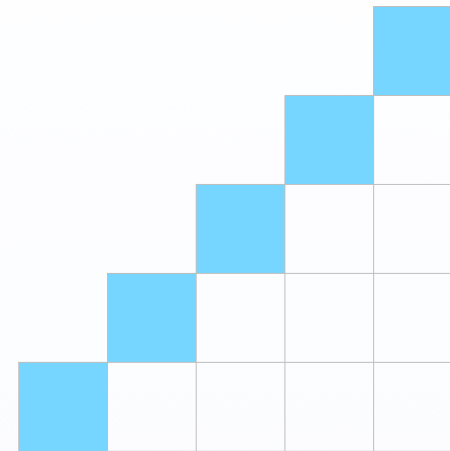
x : inkrement

y: inkrement oder bleibe gleich



Magische Bedingung

- Für den 1 Oktant
- Für jedes aufsteigende x wird ein Pixel gezeichnet
- Gehe entlang der Linie von Links nach Rechts
- Gehe dabei „eventuell“ ein Pixel hoch

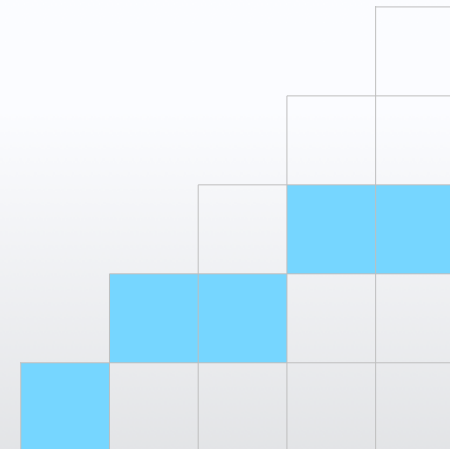
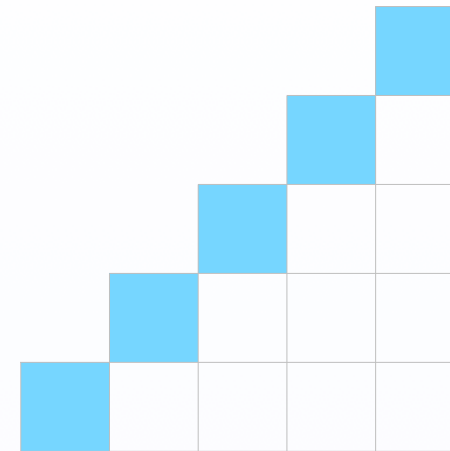


Magische Bedingung

- Für den 1 Oktant
- Für jedes aufsteigende x wird ein Pixel gezeichnet
- Gehe entlang der Linie von Links nach Rechts
- Gehe dabei „eventuell“ ein Pixel hoch

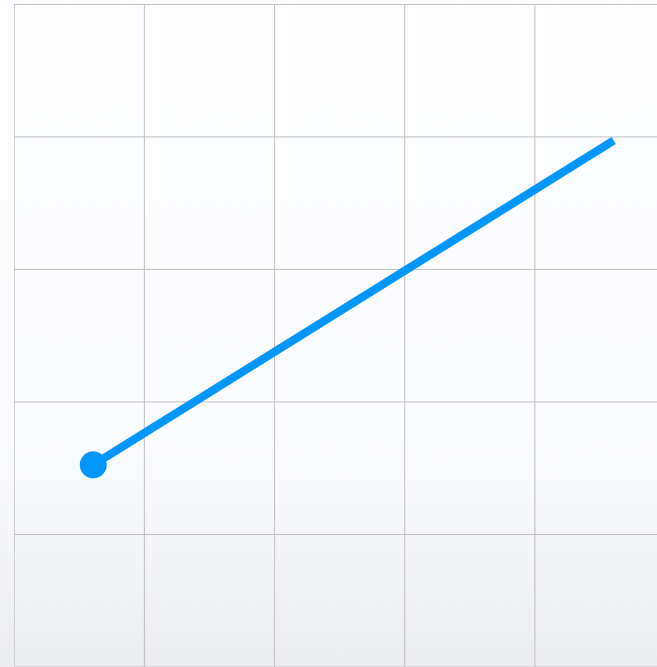
```
int y=y1;  
for(int x=x1; x <= x2 ; x++){  
    setPixel(x,y);  
    if (Bedingung)  
        y=y+1;  
    // sonst bleibt y  
}
```

wie sieht die Bedingung aus?



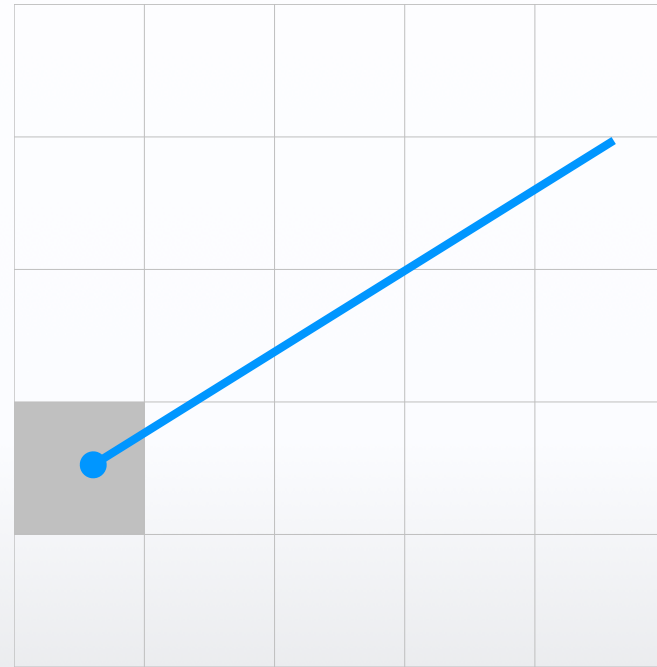
Midpoint-line Algorithmus

- Die Idee für die Bedingung



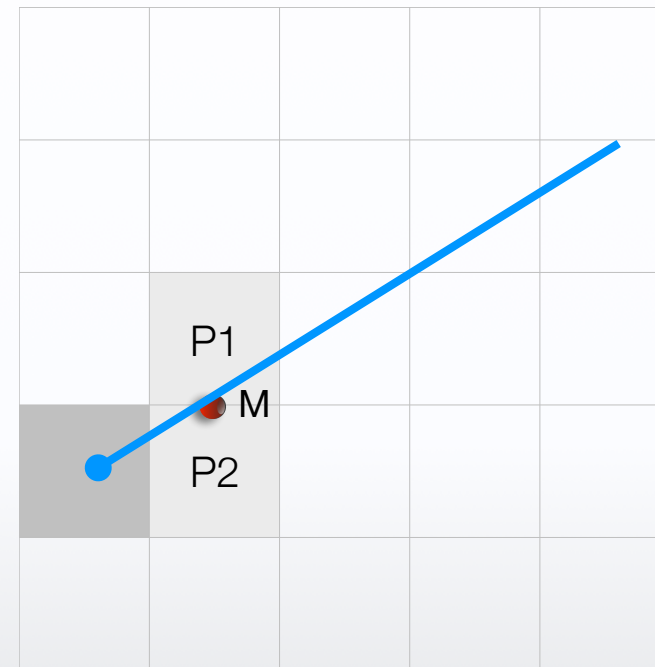
Midpoint-line Algorithmus

- Die Idee für die Bedingung
- Setze Anfangspixel



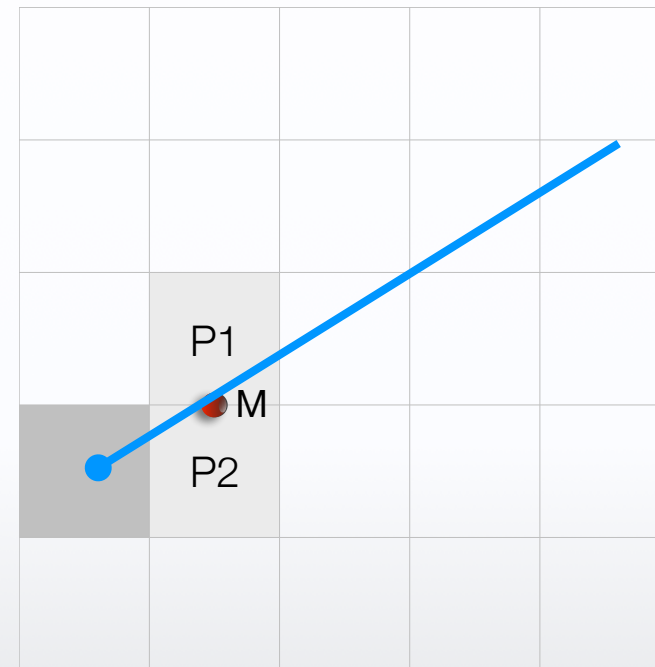
Midpoint-line Algorithmus

- Die Idee
- mögliche Pixel
- P1: $(x+1, y+1)$
- P2 : $(x+1, y)$
- Die Mitte wählen M : $(x+1, y+0.5)$



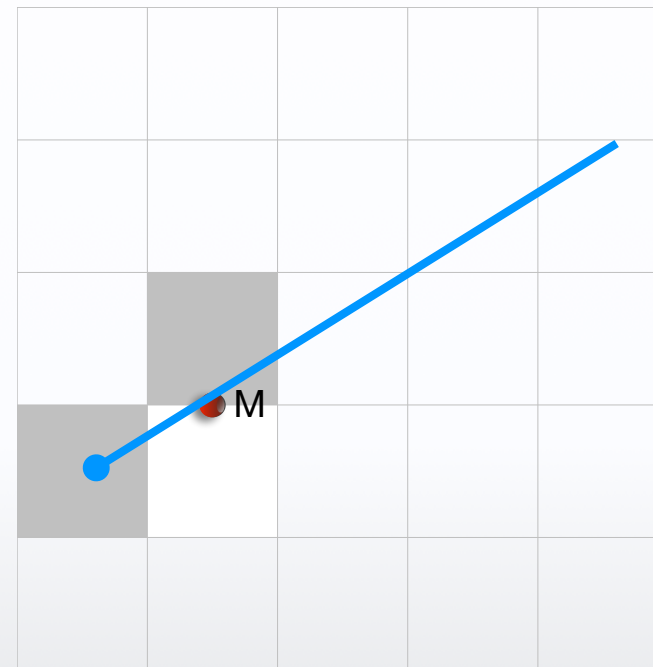
Midpoint-line Algorithmus

- Ist die Linie
- Überhalb von M
- dann: wähle P1
- Unterhalb von M
- dann: wähle P2



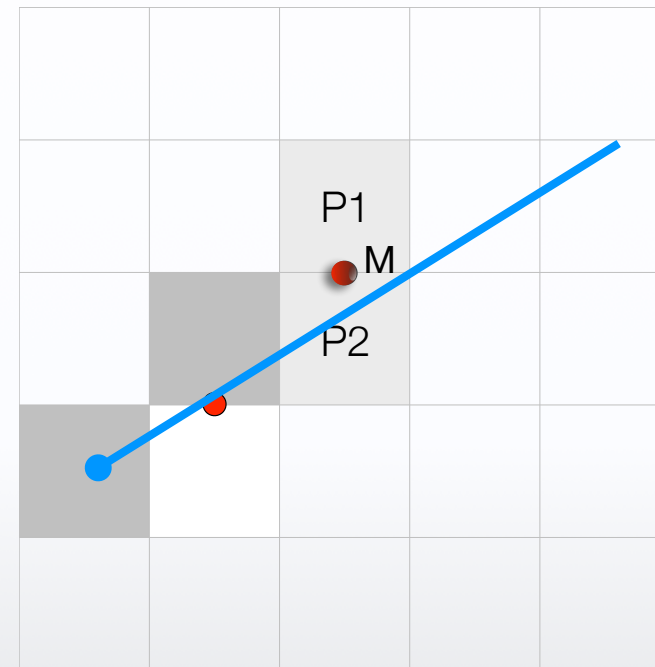
Midpoint-line Algorithmus

- Neues Pixel gesetzt
- Suche nächste mögliche Pixel
- Suche Mittelpunkt M



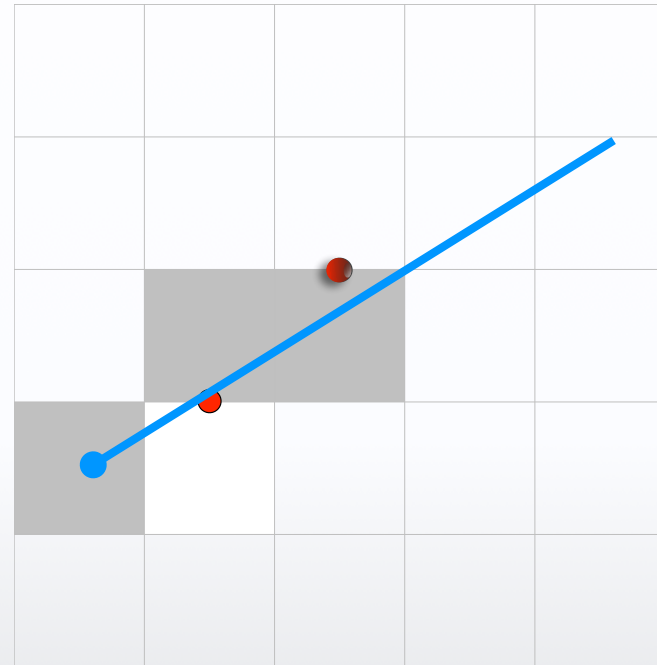
Midpoint-line Algorithmus

- Ist die Linie
- Überhalb von M
- dann: wähle P1
- Unterhalb von M
- dann: wähle P2



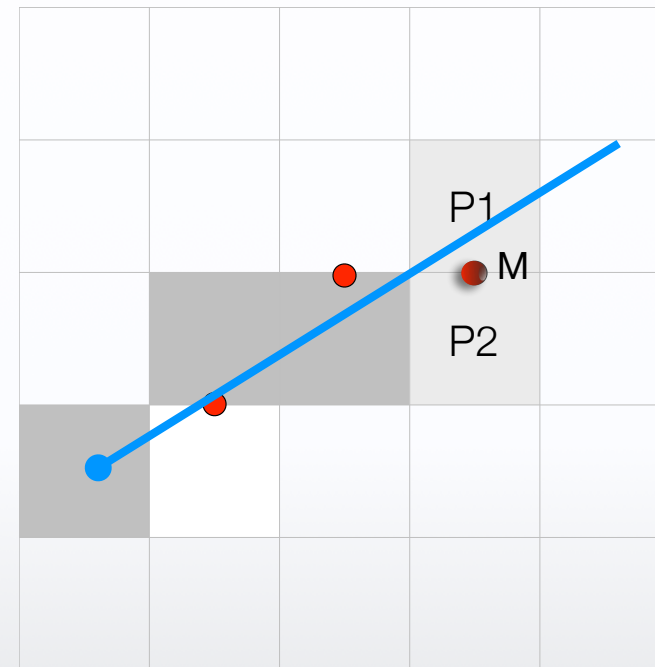
Midpoint-line Algorithmus

- Neues Pixel gesetzt
- Suche nächste mögliche Pixel
- Suche Mittelpunkt M



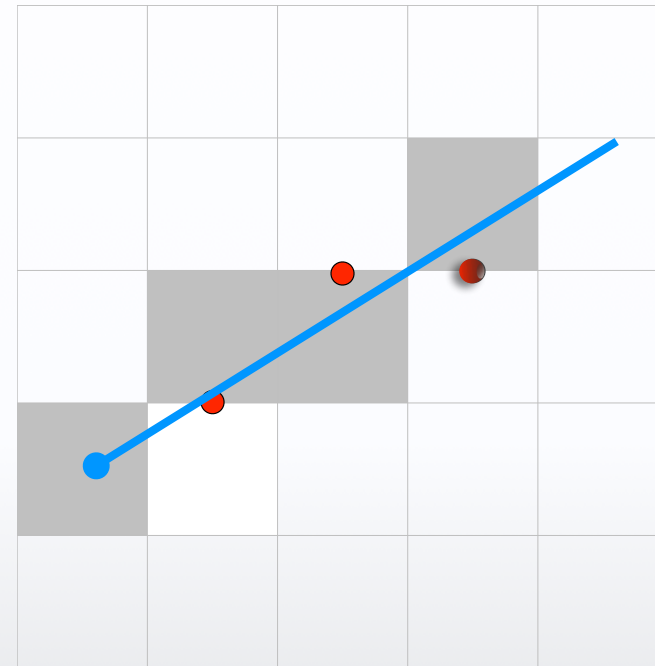
Midpoint-line Algorithmus

- Ist die Linie
- Überhalb von M
- dann: wähle P1
- Unterhalb von M
- dann: wähle P2



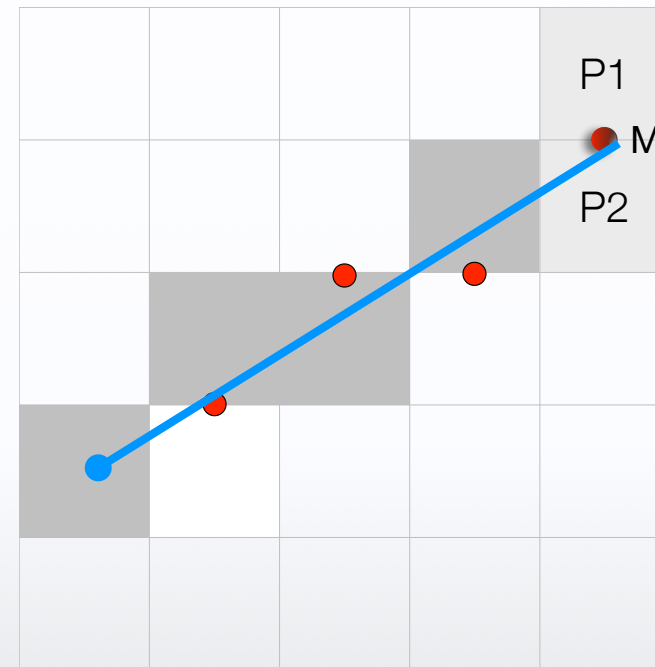
Midpoint-line Algorithmus

- Neues Pixel gesetzt
- Suche nächste mögliche Pixel
- Suche Mittelpunkt M



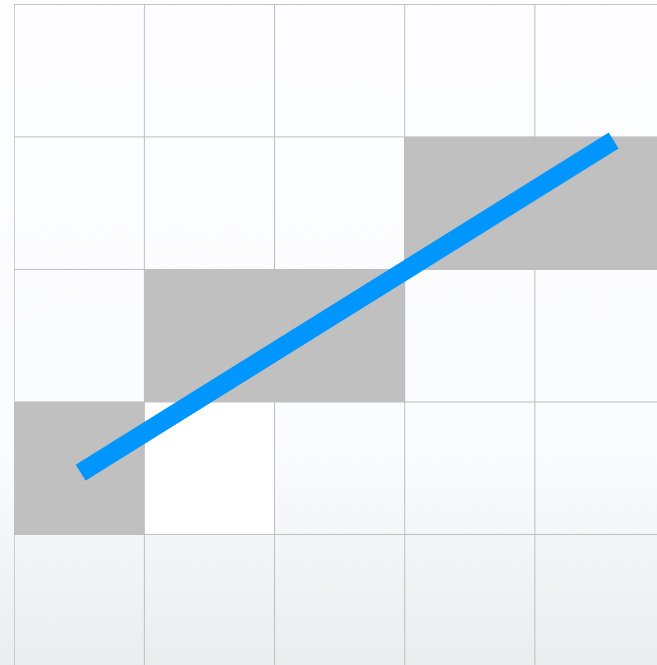
Midpoint-line Algorithmus

- Ist die Linie
- Überhalb von M
- dann: wähle P1
- Unterhalb von M
- dann: wähle P2



Midpoint-line Algorithmus

- Ist die Linie
- Überhalb von M
- dann: wähle P1
- Unterhalb von M
- dann: wähle P2



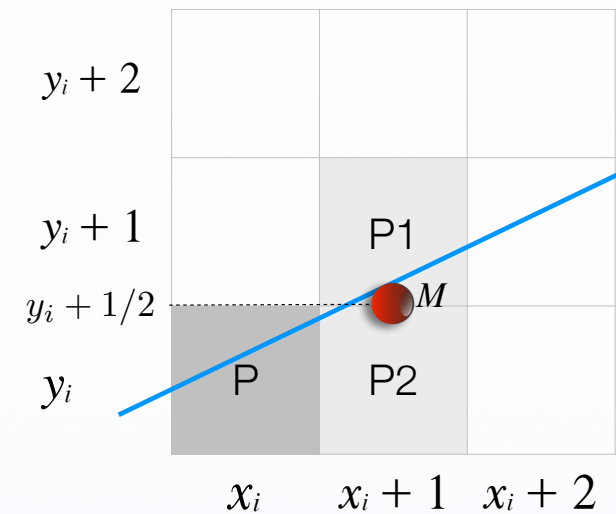
Midpoint-line Algorithmus „Bresenham '65“

- Bedingung: Punkt P gesetzt
- welcher Punkt ist der nächste?
- in Frage kommende Punkte P1, oder P2
- M = Mittelpunkt zwischen P1 & P2
- $M = (x_i + 1, y_i + 1/2)$
- IDEE:

■ Koordinaten M in die Geradengleichung einsetzen

- Gerade ist überhalb M , nehme P1
- Gerade ist unterhalb M , nehme P2
- M auf der Gerade, nehme P1 oder P2

■ Geradengleichung: $ax + by + c = 0$?



Midpoint line Algorithmus

■ Geradengleichung $ax + by + c = 0$

■ Koordinaten von M einsetzen:

■ $a(x_i + 1) + b(y_i + 1/2) + c \geq 0$

■ M ist im oberen Halbraum

■ Linie über M ; Wähle P1

$y=y+1$

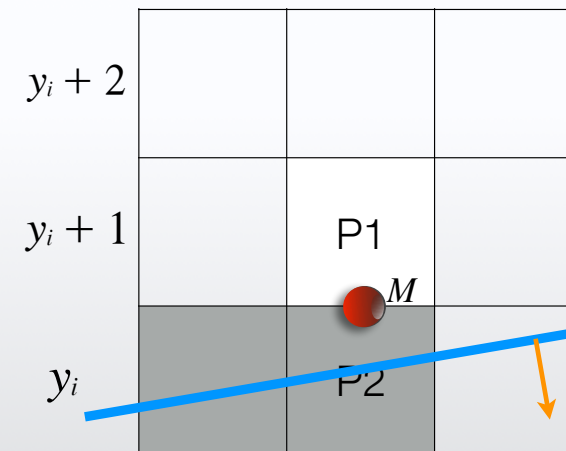
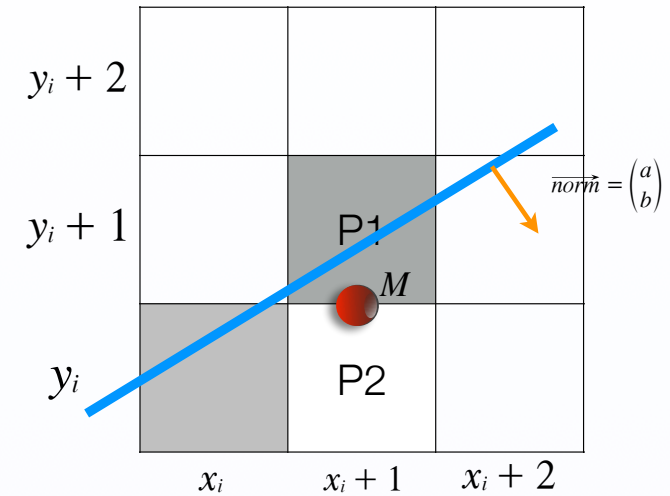
■ $a(x_i + 1) + b(y_i + 1/2) + c < 0$

■ M ist im unteren Halbraum

■ Linie unter M ; Wähle P2

$y=y$

? Wie werden a, b, c berechnet



Midpoint line Algorithmus

- Geradengleichung $ax + by + c = 0$
- Berechnung von a, b, c : (gegeben seien Start- und Endpunkt)
- Steigungsform: $y = m \cdot x + n = \frac{\Delta y}{\Delta x} \cdot x + n$
- Umstellen: $y \cdot \Delta x = \Delta y \cdot x + n \cdot \Delta x$

$$\Delta y \cdot x - \Delta x \cdot y + \Delta x \cdot n = 0$$

$$ax + by + c = 0 \Leftrightarrow \Delta yx - \Delta xy + \Delta xn = 0$$

- Ergibt: $a = \Delta y$
 $b = -\Delta x$
 $c = \Delta x \cdot n$

Nochmal überlegen

- Geradengleichung $ax + by + c = 0$
- Zunächst die festen Variablen berechnen:

$$a = \Delta y$$

$$b = -\Delta x$$

$$c = \Delta x \cdot n \quad \text{mit} \quad n = \frac{y_1 x_2 - y_2 x_1}{x_2 - x_1}$$

- Dann für jeden Pixel die Geradengleichung ausrechnen

$$a(x_i + 1) + b(y_i + 1/2) + c \geq 0$$

$y=y+1$

$$a(x_i + 1) + b(y_i + 1/2) + c < 0$$

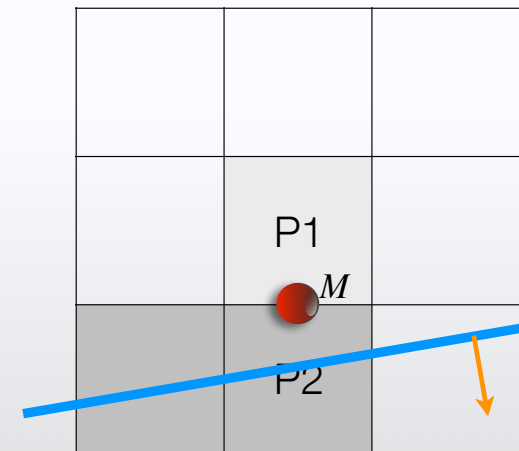
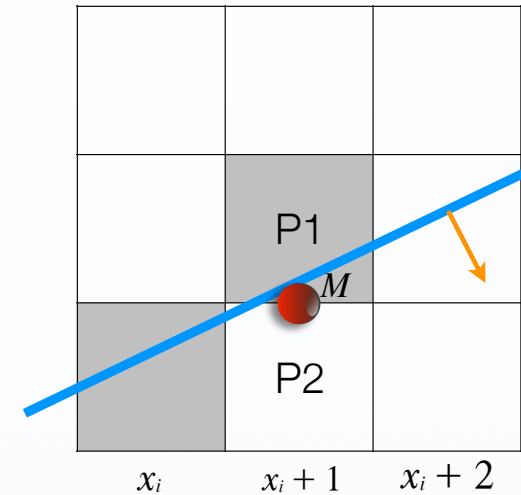
$y=y$

- Das Pixel danach mit den Koordinaten:

$$a(x_i + 2) + b(y_i + 3/2) + c \geq 0$$

...

Immer noch kein inkrementaler Vorgang



Bresenham Line Algorithm $m < 1$

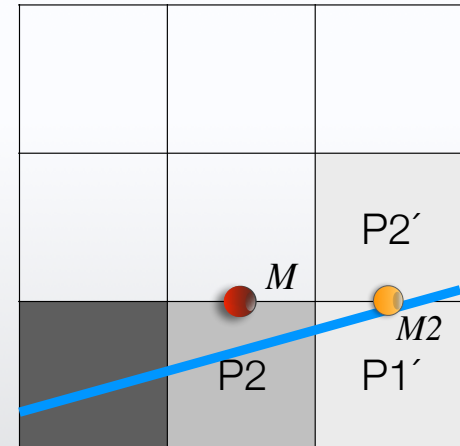
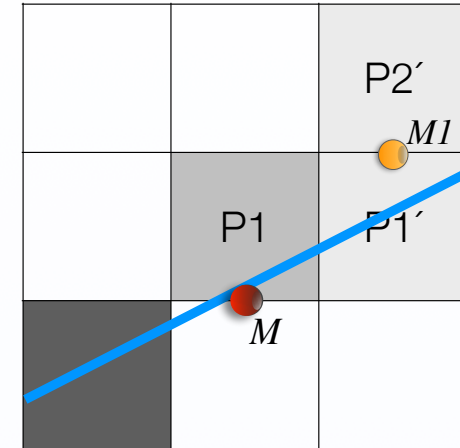
- Inkrementeller Aufbau
- Prinzip Backup-System
- Was hat sich wirklich verändert?
- Nur die Änderungen in die Rechnung einbeziehen
- Fallunterscheidung:

1 P1 ist gewählt mit M

- Nächste mögliche Punkte $P1'$ und $P2'$
- Nächster Mittelpunkt ist $M1$

2 P2 ist gewählt mit M

- Nächste mögliche Punkte $P1'$ und $P2'$
- Nächster Mittelpunkt ist $M2$

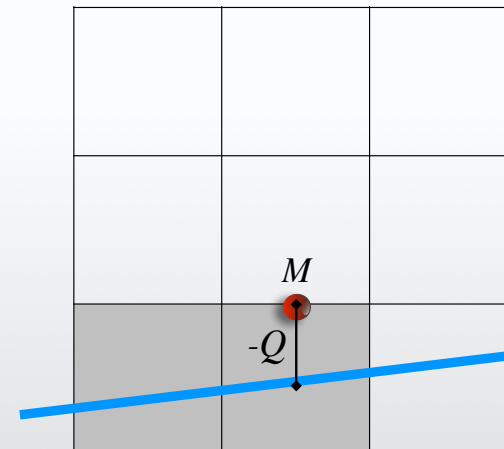
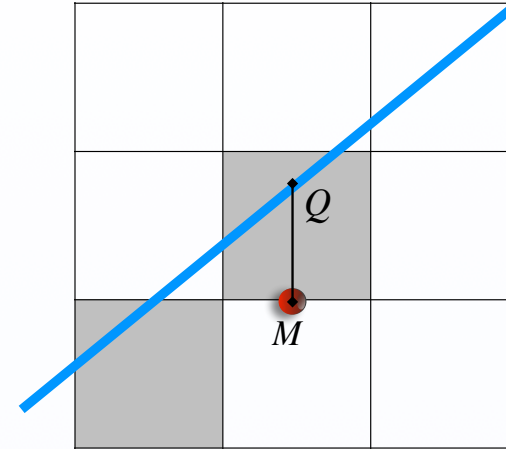


Entscheidungsvariable Q

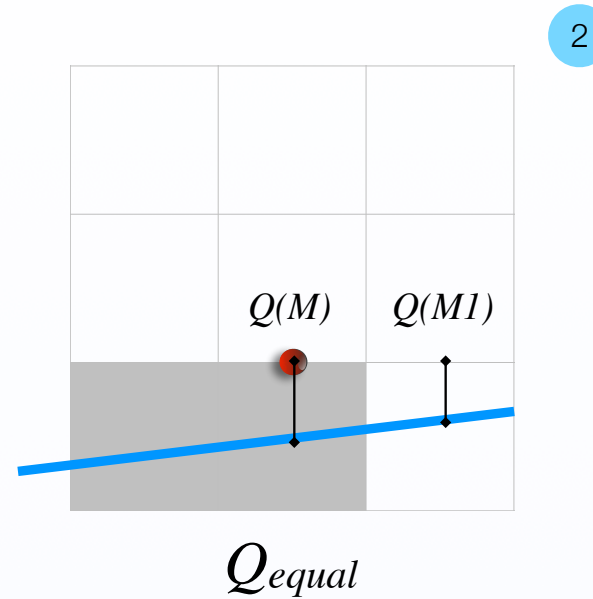
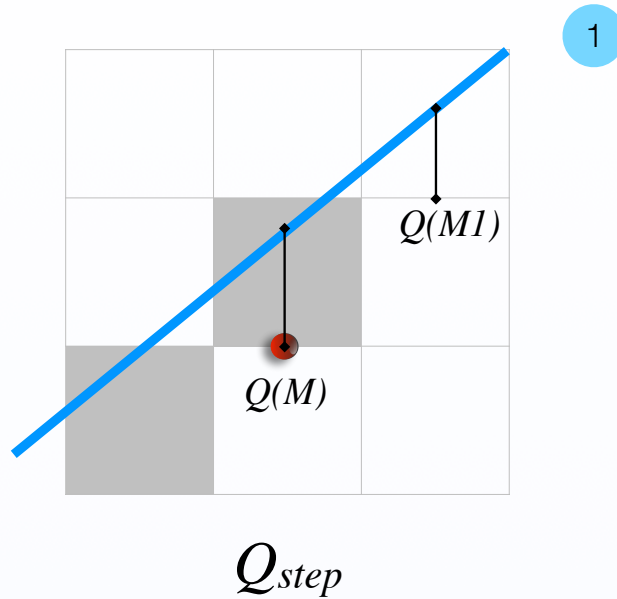
- Was kommt raus wenn die Koordinaten von M in die Geradengleichung eingesetzt werden ?

$$a(x_i + 1) + b(y_i + 1/2) + c = Q$$

- Ist Q Positiv ?
- Dann ist die Linie über dem Punkt M
- Ist Q Negativ ?
- Dann ist die Linie unter dem Punkt M
- Der Wert von Q gibt damit auch den Abstand der Linie vom Punkt M an.



Entscheidungsvariable Q



Kann $Q(M1)$ aus $Q(M)$ gebildet werden?

Was unterscheidet die Werte ?

$$Q(M1) - Q(M)$$

Bresenham (inkrement) 1

- P1 ist gewählt mit M

- Geradengleichung ist positiv

$$Q(M) \quad a(x_i + 1) + b(y_i + 1/2) + c \geq 0$$

- Dann ist $M1$ der nächste Mittelpunkt
- Geradengleichung mit $M1$ -Koordinaten.

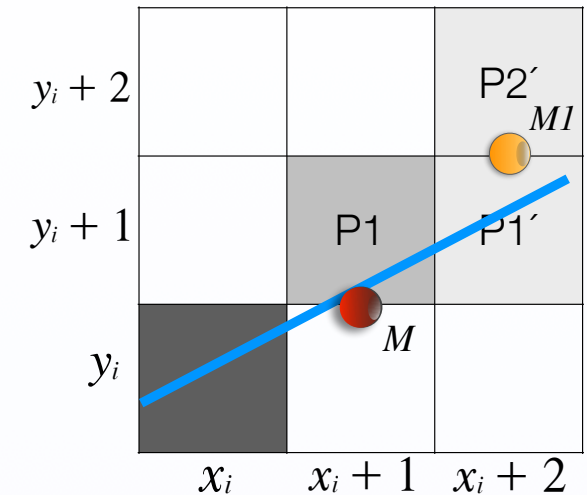
$$Q(M1) \quad a(x_i + 2) + b(y_i + 3/2) + c \quad ? \quad 0$$

- Differenz:

$$a(x_i + 2) + b(y_i + \frac{3}{2}) + c - a(x_i + 1) - b(y_i + \frac{1}{2}) - c = a + b$$

- c und damit n fallen weg! ($c = \Delta x \cdot n$)

$$Q_{step} = Q(M1) - Q(M) = a + b$$



Bresenham (inkrement)

1

Entscheidungsvariable Q inkrementel berechnen $Q_{step} = a + b$

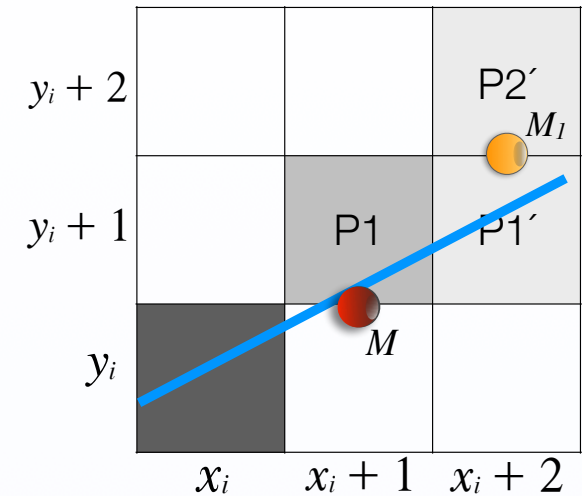
mit $a = \Delta y$, $b = -\Delta x$ $\longrightarrow$ $Q_{step} = \Delta y - \Delta x$

Daraus ergibt sich für jedes neue zu aktualisierende Q im Falle 1:

Im Falle das Q positiv ist

$$Q = Q + Q_{step}$$

```
if Q (positiv) {  
  select P1;  
  ...  
  Q = Q + Q_step;  
}
```



Bresenham (gleichbleibend) 2

- P2 ist gewählt mit M

- Geradengleichung ist negativ

$$Q(M) \quad a(x_i + 1) + b(y_i + 1/2) + c < 0$$

- Dann ist M_I der nächste Mittelpunkt
- Geradengleichung mit M_I -Koordinaten.

$$Q(M_I) \quad a(x_i + 2) + b(y_i + 1/2) + c \quad ? \quad 0$$

- Differenz:

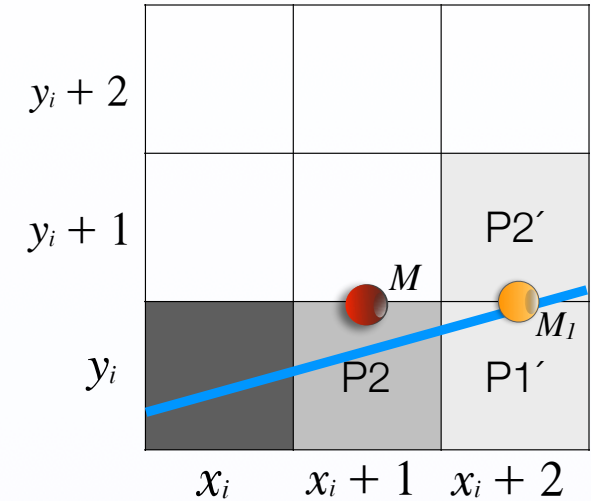
$$a(x_i + 2) + b(y_i + \frac{1}{2}) + c - a(x_i + 1) - b(y_i + \frac{1}{2}) - c = a$$

$$Q_{equal} = Q(M_I) - Q(M) = a$$

- Mit $a = \Delta y$

- Aktualisiere Q

$$Q = Q + Q_{equal}$$

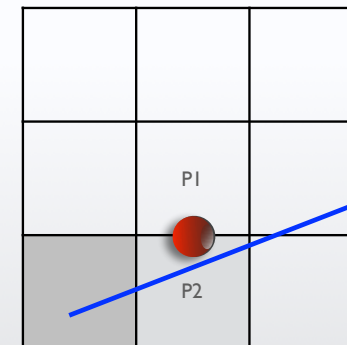
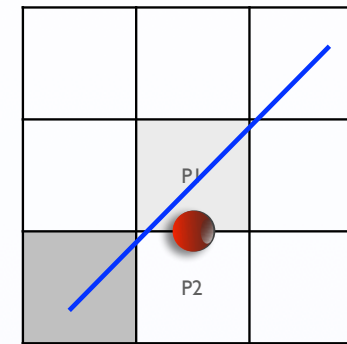


```
if Q (negativ) {  
    select P2;  
    ...  
    Q= Q+Q_equal ;  
}
```


Algorithmus

- Setze den ersten Pixel
- Berechne Entscheidungsvariable Q für aktuelles Pixel
- Wiederhole bis Endpunkt erreicht ist

```
■ if (Q Positiv oder Null)  
    ■ nächstes Pixel ist P1(oben);  
    ■ x++; y++;  
    ■ Q aktualisieren:  $Q=Q+Q\_step$ ;  
  
■ else (Q negativ)  
    ■ nächstes Pixel ist P2 (unten);  
    ■ x++;           // y bleibt const  
    ■ Q aktualisieren:  $Q=Q+Q\_equal$  ○  
■ Pixel zeichnen
```



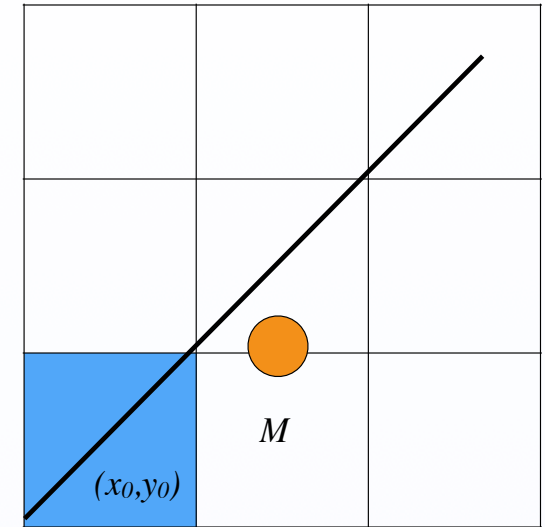
Mit welchem Initial-Wert für Q_{init} anfangen ?

Anfangswert für Q_{init}

- Startpunkt sei (x_0, y_0) :
- demnach ist: $Q_{init} = a(x_0 + 1) + b(y_0 + 1/2) + c$

$$Q_{init} = ax_0 + a + by_0 + b/2 + c$$
- da (x_0, y_0) auf der Linie liegt, ist $ax_0 + by_0 + c = 0$
- Initialwert damit:

$$Q_{init} = a + \frac{1}{2} \cdot b$$



Code - bishier

```
void draw_line(int x1, int y1, int x2, int y2){
    int y = y1
    int a = y2-y1;           // delta y
    int b = -(x2-x1);        // - delta x
    float Q = a+0.5*b;        // Initial
    float Q_step = a+b;
    float Q_equal = a;

    for (int x=x1; x<=x2 ; x++) {
        setPixel(x,y);
        if (Q<0)
            Q=Q+Q_equal;      // no y increment
        else {
            Q=Q+Q_step;        // Q >0
            y++;
        }
    }
}
```

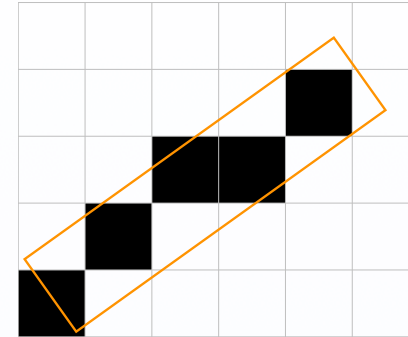
Anfangswert für Q

- Initialwert ist immer noch ein float Wert : $Q = a + \frac{1}{2}b$
- Da uns nur Vorzeichen interessiert : $Q = 2a + b$

```
void draw_line(int x1, int y1, int x2, int y2){  
  
    int y=y1  
    int a = y2-y1;           // delta y  
    int b = -(x2-x1);        // - delta x  
    int Q_init= 2*a+b;  
    int Q_step = 2*(a+b);  
    int Q_equal = 2*a;  
  
    for(int x=x1; x<=x2 ; x++) {  
        setPixel(x,y);  
        if (Q<0)  
            Q=Q+Q_equal;  
        else {  
            Q=Q+Q_step;  
            y++;  
        }  
    }  
}
```

Bresenham

- Ziel: sehr schnell eine Linie zeichnen
- Helligkeit, Stärke möglichst gleich erscheinend
- Endpunkte genau
- Schnelligkeit: Anwendung von Integer-Operationen
- Nur für 1 Oktanten zeichnen, andere Linien mit Symmetriebedingungen !!!
- Inkrementel den x-Wert erhöhen
- Mit einer Entscheidungsvariable „ Q “ den y-Wert um 1 erhöhen, oder nicht
- Entscheidungsvariable aktualisieren



Einen Strich zeichnen

- Linie zwischen

- (20/10) und (30/18)

- Berechne Konstanten:

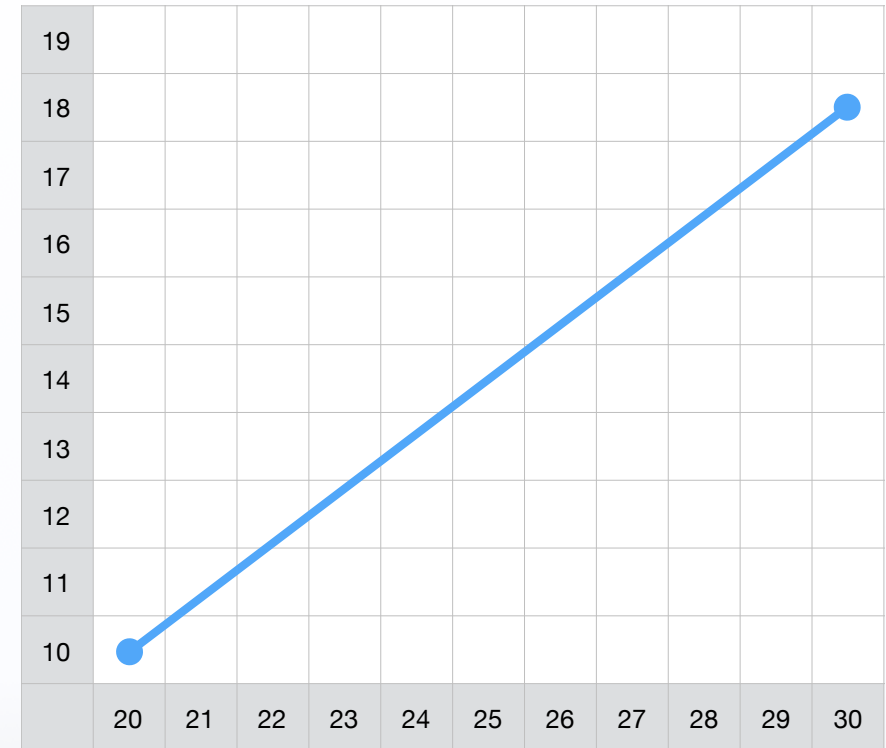
$$\Delta y = y_2 - y_1 = 18 - 10 = 8$$

$$\Delta x = x_2 - x_1 = 30 - 20 = 10$$

$$Q_{init} = 2 \cdot \Delta y - \Delta x = 16 - 10 = 6 +$$

$$Q_{step} = 2 \cdot \Delta y - 2 \cdot \Delta x = -4$$

$$Q_{equal} = 2 \cdot \Delta y = 16$$



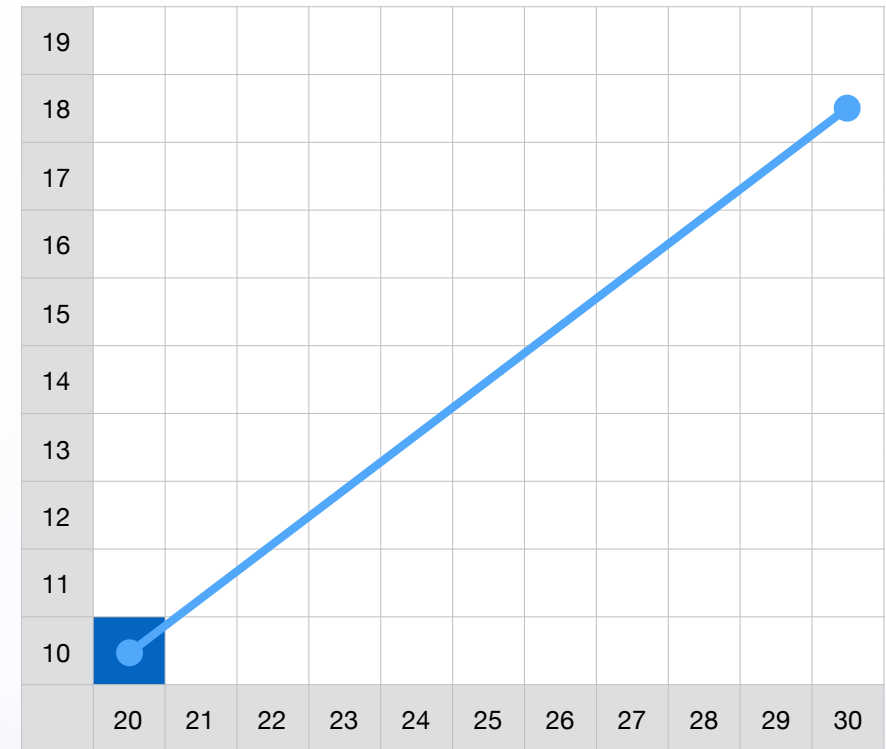
$$Q_{equal} = 16$$

$$Q_{step} = -4$$

```
if (Q < 0)
    Q = Q + Q_equal;
else {
    Q = Q + Q_step;
    y++; }
```

Einen Strich zeichnen

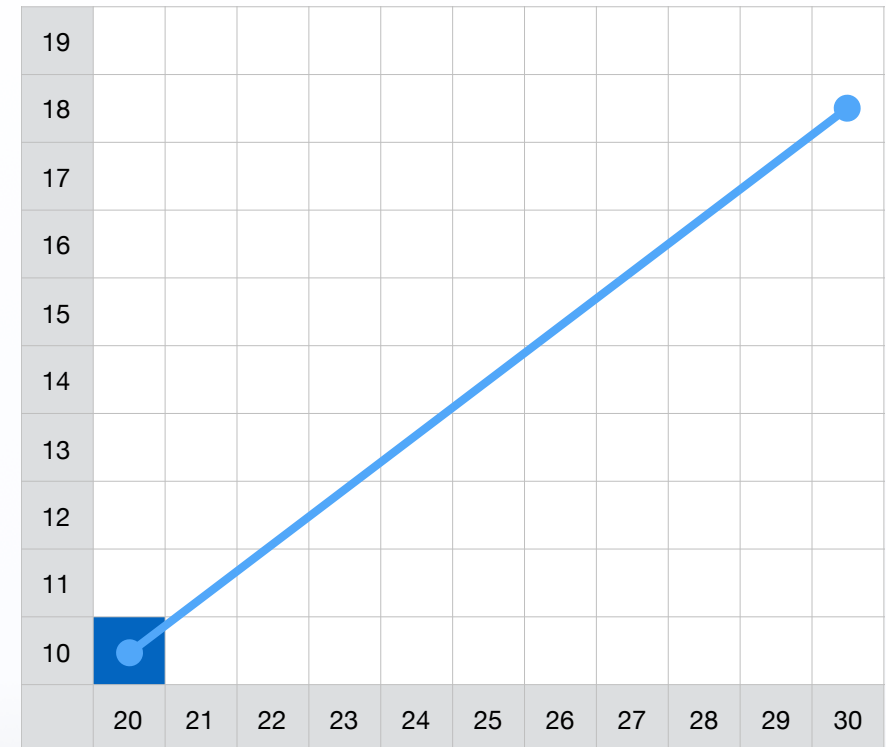
- Linie zwischen
 - (20/10) und (30/18)
- Setze Startpixel (20/10)



Einen Strich zeichnen

- 1 Schritt: $Q_{init} = 6$
- Welches ist das nächste Pixel?
- $Q(\text{update})$?

Q	Folgepixel	$Q(\text{update})$	Pixel



$$Q_{equal} = 16$$

$$Q_{step} = -4$$

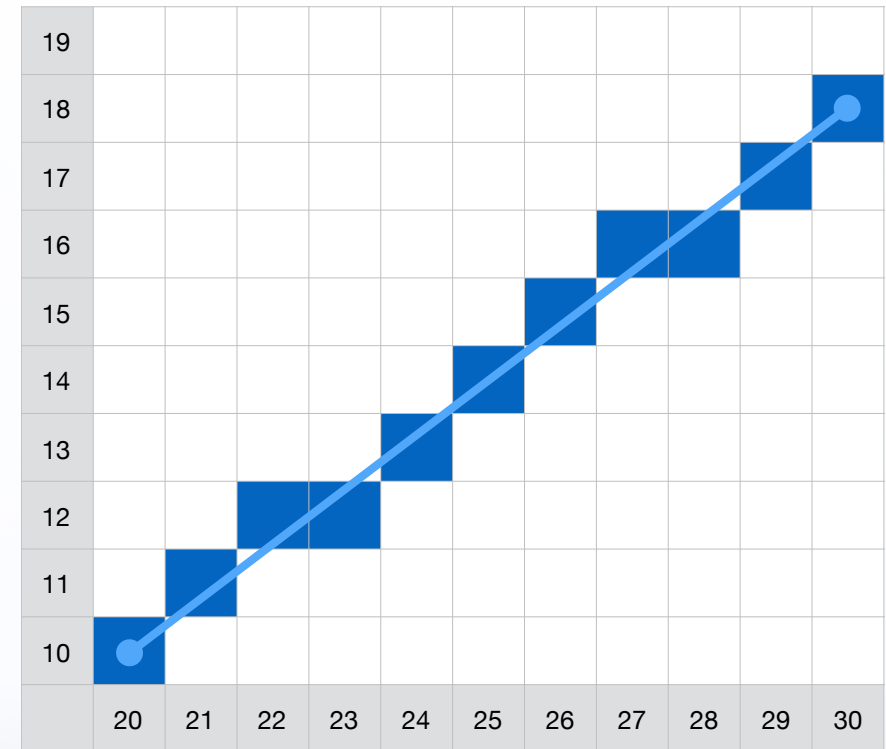
```

if (Q<0)
    Q=Q+Q_equal;
else {
    Q=Q+Q_step;
    y++; }
    
```


Einen Strich zeichnen

■ ...

Q	Folgepixel	$Q_{\text{(update)}}$	Pixel
6	x++, y++	2	(21,11)
2	x++, y++	-2	(22,12)
-2	x++, y	14	(23,12)
14	x++, y++	10	(24,13)
10	x++, y++	6	(25,14)
6	x++, y++	2	(26,15)
2	x++, y++	-2	(27,16)



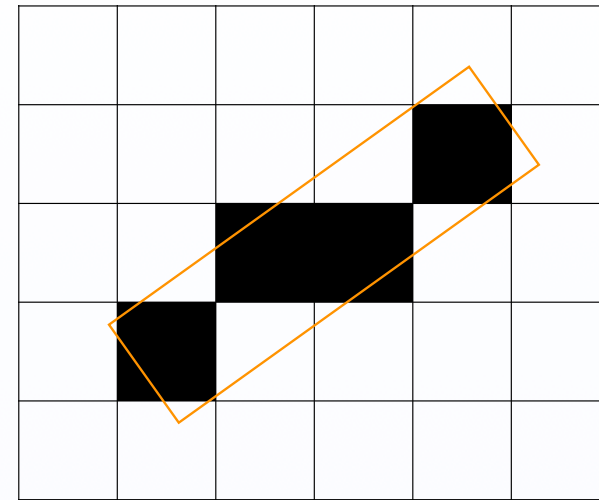
$$Q_{\text{equal}} = 16$$

$$Q_{\text{step}} = -4$$

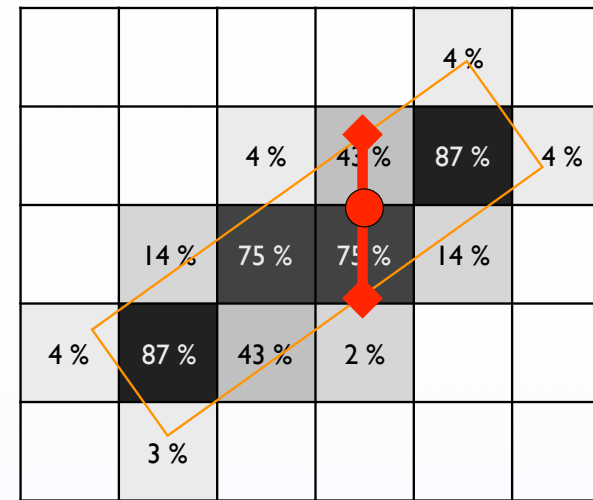
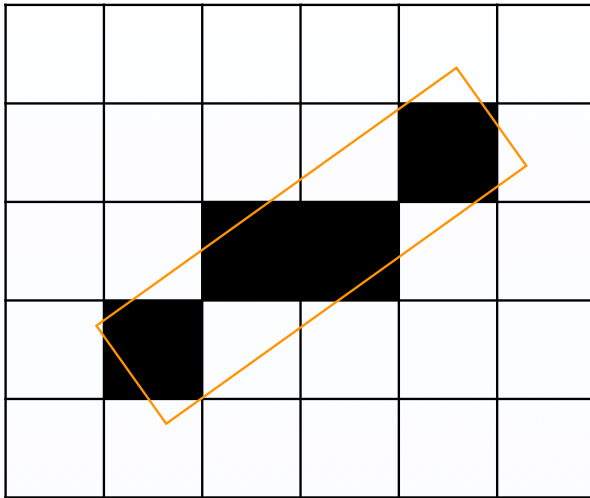
```
if (Q<0)
    Q=Q+Q_equal;
else {
    Q=Q+Q_step;
    y++; }
```

(Anti-)Aliasing

- Ergebnis von Bresenham
- Schnell, aber Problem der Treppenstufen
- Unterabtastung
 - kontinuierliche Linie durch diskrete Flächen abbilden
- Entscheidung für das Setzen oder Nichtsetzen erfolgt quasi binär (EINS/NULL)



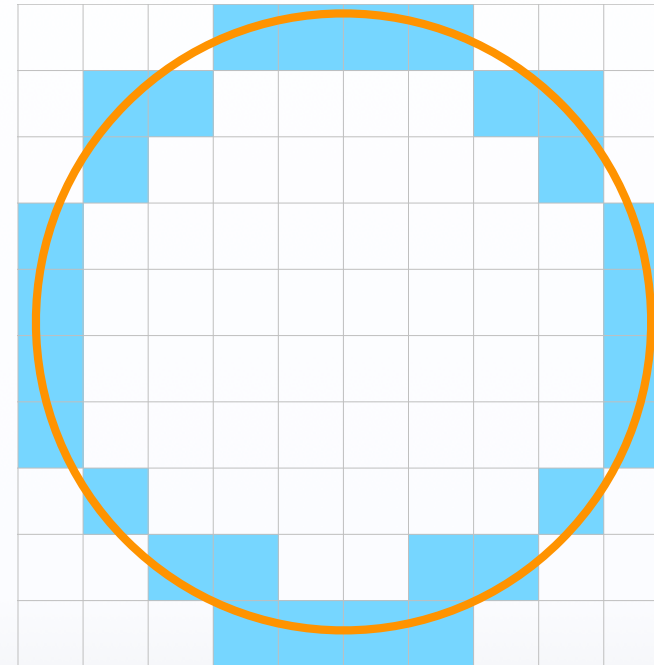
(Anti-)Aliasing



- Helligkeit eines Pixels in Abhängigkeit von der Verdeckung des Pixels durch die Linie
- Wie berechnen wir einen Verdeckungswert ?
- Für jedes x, nicht ein y-Wert, sondern z.B. drei Werte (Oberhalb, Pixel, Unterhalb)
- Aktuelle Hardware:
 - Bild wird in doppelte/vierfache Auflösung berechnet, und runtergerechnet (Supersampling)

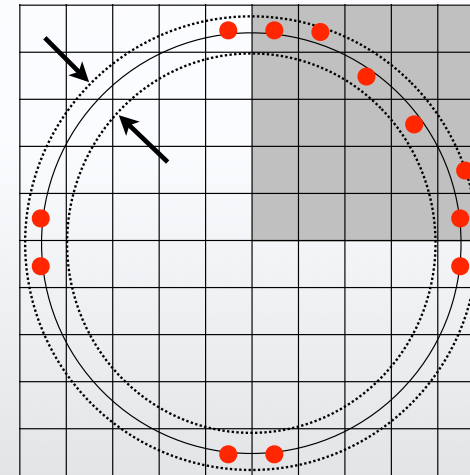
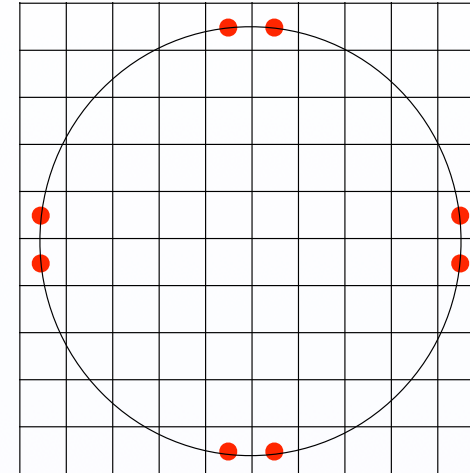
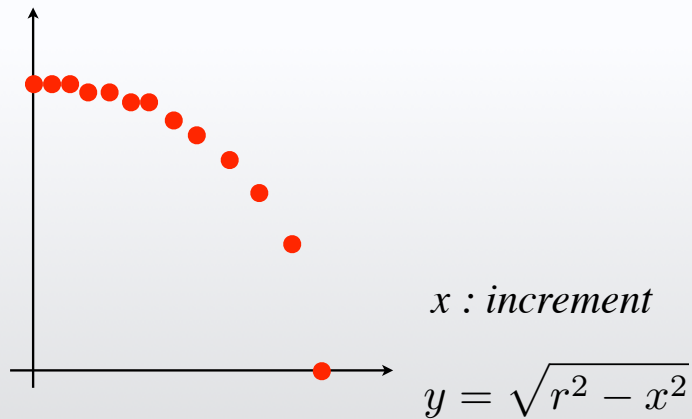
Scanline Conversion

- Linien rastern
 - naive Methode
 - Digital Differential Analyzer
 - Bresenham Algorithmus
- Kreise rastern
 - Quadratische Methode
 - Trigonometrische Methode
 - Symmetrien ausnutzen
 - Midpoint Algorithmus



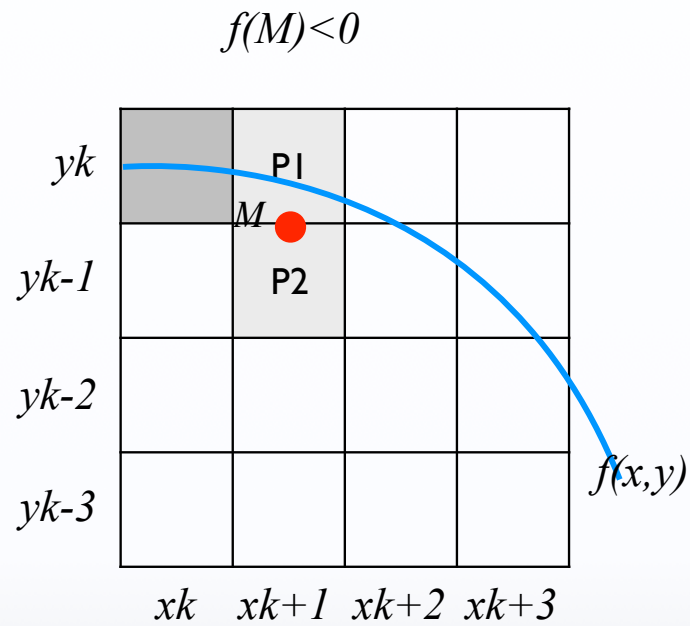
Kreisbeschreibung

- Finde alle Pixel auf dem Radius $x^2 + y^2 - r^2 = 0$
- Finde alle Pixel fast auf dem Kreis $|x^2 + y^2 - r^2| < \delta$
- Problem: bei kontinuierlicher Schrittweite entstehen “gaps”

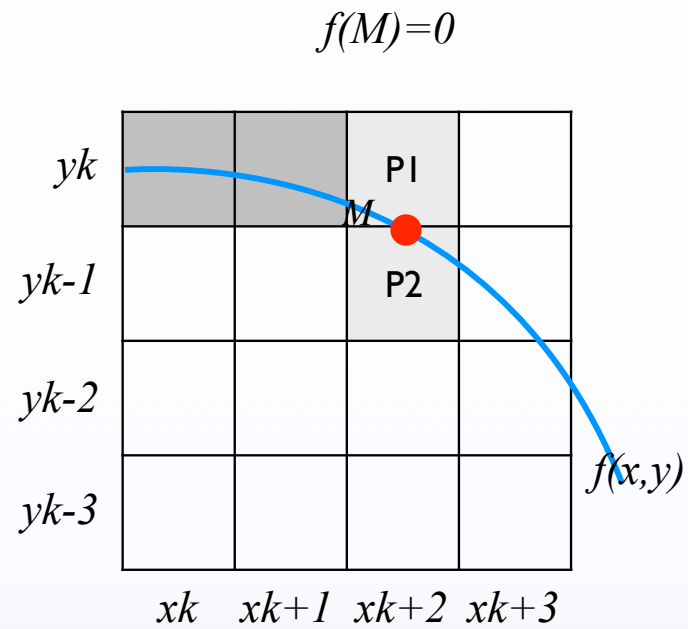


Midpoint Circle Algorithmus

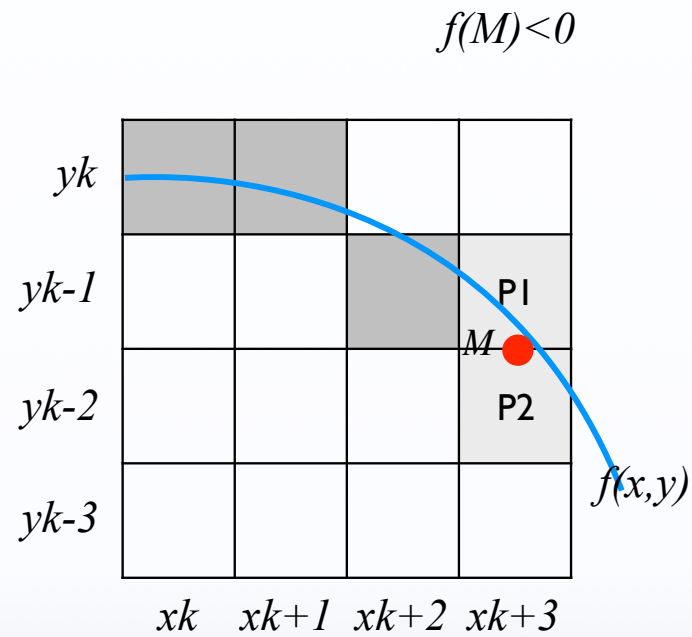
Analoge Überlegungen zu Bresenham



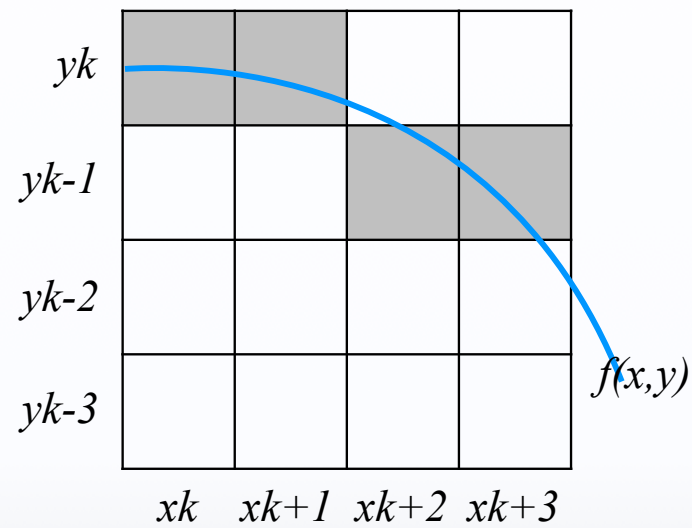
Midpoint Circle Algorithmus



Midpoint Circle Algorithmus



Midpoint Circle Algorithmus



Midpoint Circle Algorithmus

- Inkrementaler Vorgang: $f(x,y) = x^2 + y^2 - r^2$

$$f(x,y) \begin{cases} < 0 & \text{innerhalb} \\ = 0 & \text{auf dem Kreis} \\ > 0 & \text{ausserhalb} \end{cases}$$

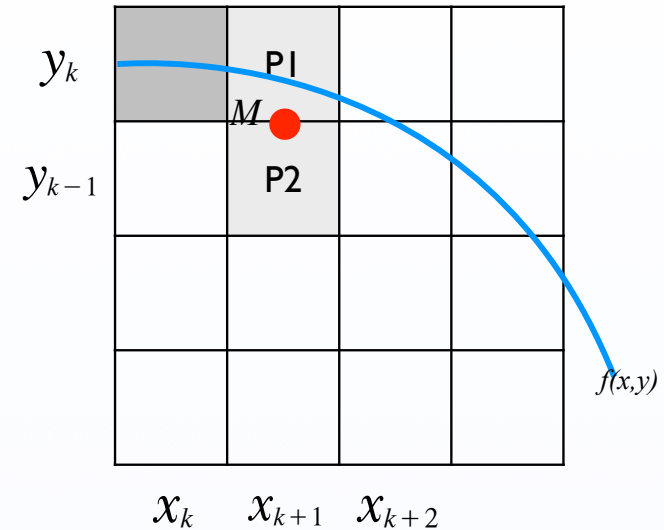
- mögliche Pixel P1, oder P2

- wähle M als Mittelpunkt

$$f(M) = f(x_{k+1}, y_k - \frac{1}{2})$$

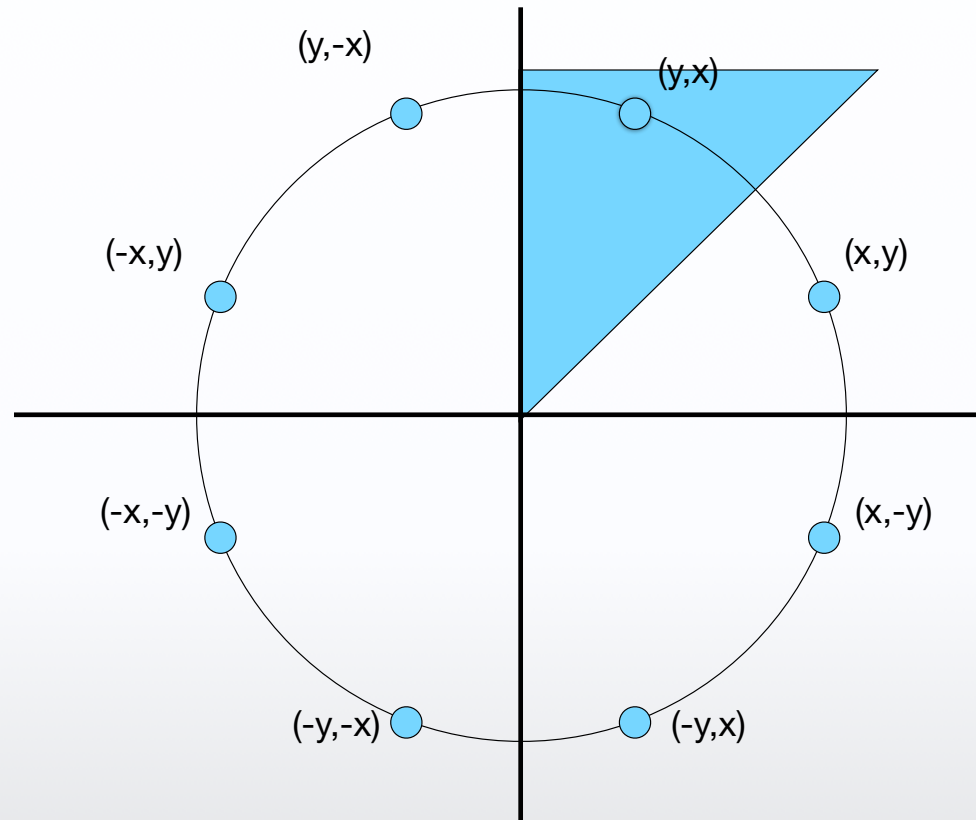
$$f(M) = (x_k + 1)^2 + (y_k - \frac{1}{2})^2 - r^2$$

- $Q > 0$, wähle P2 ; inkrementel: $x+1; y-1$
- $Q < 0$, wähle P1 ; inkrementel: $x+1$
- Entscheidungsvariable Q



Erweiterung auf Kreise

- Kreise sind symmetrisch
- Berechnung für einen Oktant
- 8'er Symmetrie



MidpointCircle code

```
void circleMidpoint(int a, int b, int r) {  
  
    int x=0;  
    int y=r;  
    int Q= 1-r; // nur bei integer radius  
    circleSymmetryPlot(a,b,x,y);    // Octant  
  
    while (x<y) {  
        x++;  
        if(Q<0)  
            Q+=2*x+3;  
        else{  
            y--;  
            Q+=2*(x-y)+5;  
        }  
        circleSymmetryPlot(a,b,x,y);  
    }  
}
```

